

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Curso Académico 2023/2024

Trabajo Fin de Grado

Definición de Guías de Diseño para Bases de Datos Orientadas a Grafos

Autor: José Javier Martín Hernández

Directores:
Belén Vela Sánchez
José María Cavero Barca

RESUMEN

Las bases de datos relacionales son utilizadas en el desarrollo de software y han demostrado su eficacia en varios contextos para el almacenamiento de información a lo largo de las distintas décadas.

Sin embargo, con el paso del tiempo, los sistemas de información deben enfrentarse a la manipulación de grandes cantidades de datos. Tradicionalmente, se ha usado bases de datos relacionales, pero posteriormente, han surgido las bases de datos NoSQL que se han diseñado para manipular grandes volúmenes de datos de manera muy rápida.

Como se indica en [1], las bases de datos NoSQL están diseñadas específicamente para manejar aplicaciones Web y grandes volúmenes de datos. Por esta razón son utilizadas, entre otras, por empresas como Google, Twitter, LinkedIn, Facebook, Amazon, etc.

Existen distintos tipos de base de datos NoSQL, entre las que se encuentran las bases de datos orientadas a grafos. Este tipo de bases de datos ha tenido mucho éxito en sistemas con información muy relacionada, como las redes sociales.

Las bases de datos orientadas a grafos han facilitado la exploración de los datos gracias a su naturaleza con una estructura en forma de grafo, permitiendo hacer recorridos por caminos cortos del grafo sin necesidad de verificar la totalidad de caminos del árbol de grafos. Este tipo de base de datos permite almacenar la información como nodos de un grafo y sus respectivas relaciones, permitiendo así aplicar la teoría de grafos para recorrer la base de datos.

En este trabajo nos centraremos en este tipo de bases de datos, concretamente en el sistema de gestión de BD NoSQL Neo4J, un software libre de bases de datos orientadas a grafos. En Neo4J usaremos el lenguaje Cypher, para poder crear la base de datos orientada a grafo, a través de las transformaciones de nuestro dataset de entrada.

El objetivo en esta memoria es definir un conjunto de guías para diseñar una base de datos orientada a grafos a partir de un modelo conceptual. Para ello, hemos elegido un ejemplo sencillo a partir del cual se han analizado las distintas alternativas para plantear las guías que usaremos para diseñar una base de datos orientada a grafos recogiendo la mayor semántica posible y teniendo en cuenta las consultas frecuentes a la base de datos.

Palabras clave: Bases de Datos NoSQL, Bases de Datos orientadas a Grafos, Neo4J, diseño de bases de datos, guías de transformación, dataset.

ABSTRACT

Relational databases are used in software development and have proven their effectiveness in several methods of information storage through the last decades.

However, over time, information systems have to endure the manipulation of large amounts of data. Traditionally, relational databases have been used, but subsequently, NoSQL databases have emerged that have been designed to handle large volumes of data very quickly.

As pointed out in [1], NoSQL databases are specifically designed to handle Web applications and large volumes of data. For this reason, they are used, among others, by companies such as Google, Twitter, LinkedIn, Facebook, Amazon, etc. use them.

There are different types of NoSQL databases, including graph-oriented databases. This type of database has been very successful in systems with very related information, such as social networks.

Graph-oriented databases have facilitated data exploration due to its nature with a graph-structured form, allowing us to go across short paths in the graph without the need to check all the paths in the graph tree. This type of database allows information to be stored as nodes of a graph and their respective relationships, thus allowing the application of graph theory to traverse the database.

In this work we will focus on this type of database, specifically on the Neo4J NoSQL DB management system, a free software for graph-oriented databases. In Neo4J we will use Cypher language to create a graph-oriented database through the transformations of our input dataset.

The objective of this thesis is to create a set of guidelines to design a graph-oriented database based on a conceptual model. To do so, we have chosen a simple example from which we have analyzed the different alternatives to propose the guidelines that we will use to design the graph-oriented database collecting the largest possible semantics and taking into account the frequent queries to the database.

Keywords: NoSQL databases, graph-oriented databases, Neo4J, database design, transformation guides, dataset.

ÍNDICE

RESUMEN **2**

ABSTRACT **3**

I. INTRODUCCIÓN **7**

1.1 Motivación del Trabajo 7

1.2 Objetivos 7

1.2.1 Objetivo Principal 7

1.2.2 Objetivos Parciales 8

1.3 Método de Trabajo 8

1.4 Medios Hardware y Software 9

1.5 Estructura de la Memoria 10

II. ESTUDIOS PREVIOS **11**

2.1 BD NoSQL 11

2.1.1 BD Clave-Valor 11

2.1.2 BD Orientadas a Documentos 13

2.1.3 BD Familia de Columnas 14

2.1.4 BD Orientadas a Grafos 15

2.2 Neo4J 16

III. DESARROLLO **17**

3.1 Caso de Estudio 17

3.2 Diseño de BD orientada a Grafos 18

3.2.1 Transformación de Entidades y Atributos. 18

3.2.2 Transformación Interrelación 1:1 22

3.2.3 Transformación Interrelación 1:N 26

3.2.3 Transformación Interrelación N:M 29

3.2.4 Transformación Interrelación reflexiva 32

3.2.5 Transformación Jerarquías 34
3.3 VALIDACIÓN 39
3.3.1 Transformación de la Interrelación 1:1 - Ocupa 40
3.2.2 Transformación de la Jerarquía de Empleados 43
3.3.3 Transformación de la Interrelación 1:N - Imparte 45
3.3.4 Transformación de la Interrelación N:M - Trabaja 47
3.3.5 Transformación de la Interrelación Reflexiva - Es_Compañero_De 49
3.3.6 Esquema final y consultas 50
3.4 Lecciones aprendidas 55
IV. CONCLUSIONES Y FUTUROS TRABAJOS 57
V. REFERENCIAS BIBLIOGRÁFICAS 58

I. INTRODUCCIÓN

En primer lugar, en este capítulo se presentará el trabajo y la motivación. Después se planteará el objetivo principal y los objetivos parciales a cumplir, y se descubrirá el método de trabajo, así como las herramientas utilizadas para el desarrollo del trabajo. Se terminará describiendo la estructura de la memoria del Trabajo Fin de Grado (TFG).

1.1 Motivación del Trabajo

La motivación de este trabajo es conocer un nuevo tipo de bases de datos como son las BD NoSQL, cada vez más usadas en aplicaciones web en las que necesitamos manejar de forma ágil grandes cantidades de datos.

De los distintos tipos de bases de datos NoSQL, nos centraremos en las bases de datos orientadas a grafos (BDoG), muy interesantes por ser capaz de manejar muchos datos de forma rápida y por la capacidad de establecer relaciones entre sus datos; de lo que emana su uso en aplicaciones web y redes sociales.

Como se indica en [2] no tenemos unas guías de diseño predefinidas a la hora de la creación de las bases de datos orientadas a grafos, lo que lleva a proponer unas guías de diseño propias para la creación de una base de datos de este tipo, con la motivación de mantener la mayor semántica posible para no perder información.

Diseñaremos una base de datos orientada a grafos para comprobar el rendimiento y la semántica recogida a través de consultas para validar las guías propuestas.

1.2 Objetivos

A continuación, se presenta el objetivo principal de este TFG, así como los objetivos parciales que debe seguir para alcanzar al objetivo principal.

1.2.1 Objetivo Principal

El objetivo principal del trabajo es desarrollar una serie de guías de transformación para diseñar una base de datos orientada a grafos partiendo de un esquema conceptual.

1.2.2 Objetivos Parciales

Para alcanzar el objetivo principal se han especificado los siguientes objetivos parciales:

1. Comprensión de las bases de datos NoSQL.

2. Comprensión del funcionamiento de una base de datos orientada a grafos (BDoG) y el producto Neo4J

3. Definir reglas de transformación para obtener una BD orientada a grafos a partir de un esquema conceptual.

4. Validar la propuesta.

1.3 Método de Trabajo

El método seguido en este TFG se refleja en la Figura 1, donde constan los pasos llevados a cabo para realizar el trabajo incluyendo estudios previos, desarrollo y validación de las reglas.

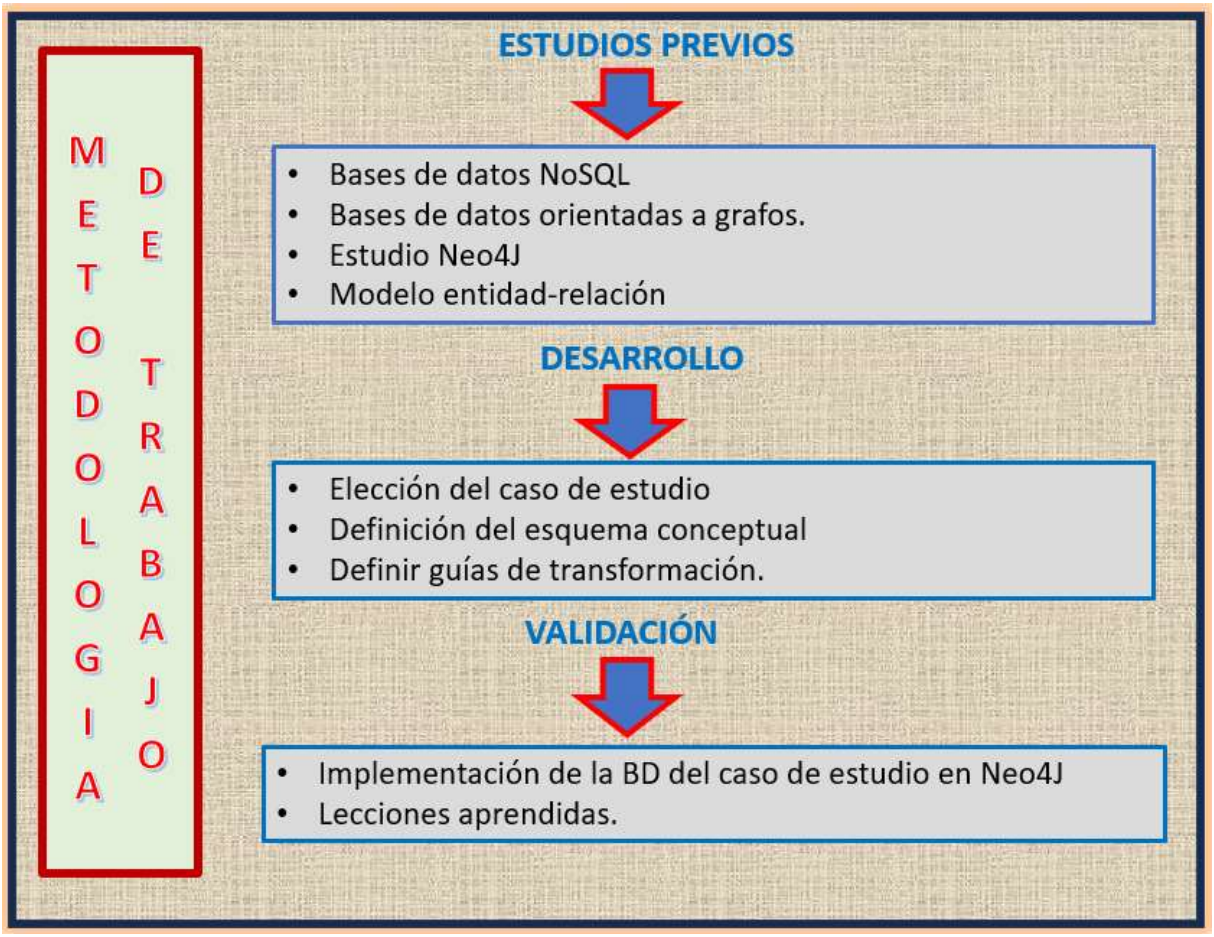


Figura 1. Método de trabajo

1.4 Medios Hardware y Software

Los medios Software utilizados para el desarrollo del trabajo son:

- Draw.io: Software para poder representar cualquier tipo de diagrama. En este caso utilizado para crear un esquema conceptual.
- PowerPoint: Software para poder realizar las figuras.
- Paint: Software para poder modificar imágenes, usadas para las figuras de la memoria.
- Word: Software para realizar la memoria del Trabajo Fin de Grado.
- Producto Neo4J: Sistema de gestión de BD orientadas a grafos, utilizando la versión 1.5.9 instalada en local.

Los medios Hardware utilizados para el desarrollo del trabajo son:

- Ordenador portátil (Samsung Galaxy Book3): Hardware donde he realizado el desarrollo de la memoria y de las figuras del Trabajo Fin de Grado.
- Ordenador personal: Hardware más potente, en el cual he realizado la creación de la base de datos en Neo4J y la validación de las guías de transformación.

1.5 Estructura de la Memoria

El presente trabajo se agrupa en los siguientes VI capítulos:

- Capítulo I: contiene la introducción del trabajo, la presentación y los objetivos.
- Capítulo II: incorpora conceptos previos, necesarios para el entendimiento del trabajo que se va a realizar.
- Capítulo III: muestra el desarrollo del trabajo, donde se presenta el caso de estudio y se proponen las guías de transformación. Además, llevaremos el caso de estudio a Neo4J para mediante consultas validar la propuesta.
- Capítulo IV: expondrá las conclusiones del trabajo rescatadas de las validaciones.
- Capítulo V: referencias bibliografía utilizadas para la realización del trabajo.

II. ESTUDIOS PREVIOS

En este capítulo vamos a definir los conceptos previos necesarios para entender mejor el desarrollo del trabajo. Comenzaremos hablando de las bases de datos NoSQL, incluyendo los distintos tipos existentes.

También hablaremos de Neo4J, un gestor de bases de datos orientadas a grafos, usando el lenguaje Cypher. Usando este lenguaje haremos la carga de un dataset para crear una BD orientada a grafos usando las guías propuestas para transformar el esquema conceptual.

2.1 BD NoSQL

Son sistemas de gestión que se alejan de las bases de datos relacionales. Como se indica en [3] las bases de datos NoSQL se crean para manejar grandes volúmenes de datos distribuidos y escalables, además de proporcionar un almacenamiento más flexible y eficiente que las bases de datos tradicionales.

Una de las características principales de las bases de datos NoSQL, es que permiten almacenar datos sin un esquema fijo, facilitando la incorporación de nuevos campos sin tener que modificar la estructura de la base de datos.

Como se indica en [4] una de las principales ventajas de este tipo de base de datos es la velocidad y la escalabilidad que proporcionan, convirtiéndose en una opción popular por su rendimiento y facilidad de uso. Estas características hacen que sean de gran utilidad en el análisis de grandes volúmenes de datos y el desarrollo de aplicaciones móviles y web.

Existen varios tipos de bases de datos NoSQL, que se resumirán a continuación: BD Clave-Valor, BD Orientada a Documentos, BD Familia de Columnas, BD Orientada a grafos.

2.1.1 BD Clave-Valor

Como se indica en [5], las bases de datos clave-valor son las más simples de las bases de datos NoSQL y son un buen punto de partida para iniciarse en este tipo de BD. Este tipo de bases de datos se basa en almacenar pares clave-valor.

Cada par consta de una clave y un valor asociado. La clave puede ser de cualquier tipo: entero, cadena de caracteres, listas de valores, etc., siendo el único requisito de diseño de una

base de datos de este tipo la unicidad de la clave, es decir, no puede repetirse la misma clave para dos pares distintos, pero sí su valor. Cada clave debe tener su valor asociado, por ello, podemos consultar la información de cualquier valor conocida su clave.

En resumen, destacamos de este tipo de base de datos la facilidad de manejar grandes cantidades de información y la capacidad de almacenar cualquier tipo de información compleja al permitir claves de todo tipo de caracteres y longitudes.

Este tipo de bases de datos se puede manejar en productos como Redis, usándose en aplicaciones Web como Netflix o Twitter a nivel de guardar registros de usuarios y estadística.

Como se observa en la Figura 2, este sería un ejemplo de este tipo de BD, donde tenemos pares clave-valor. En la imagen vemos como consultando la clave obtenemos el valor donde se almacena la información del vendedor.

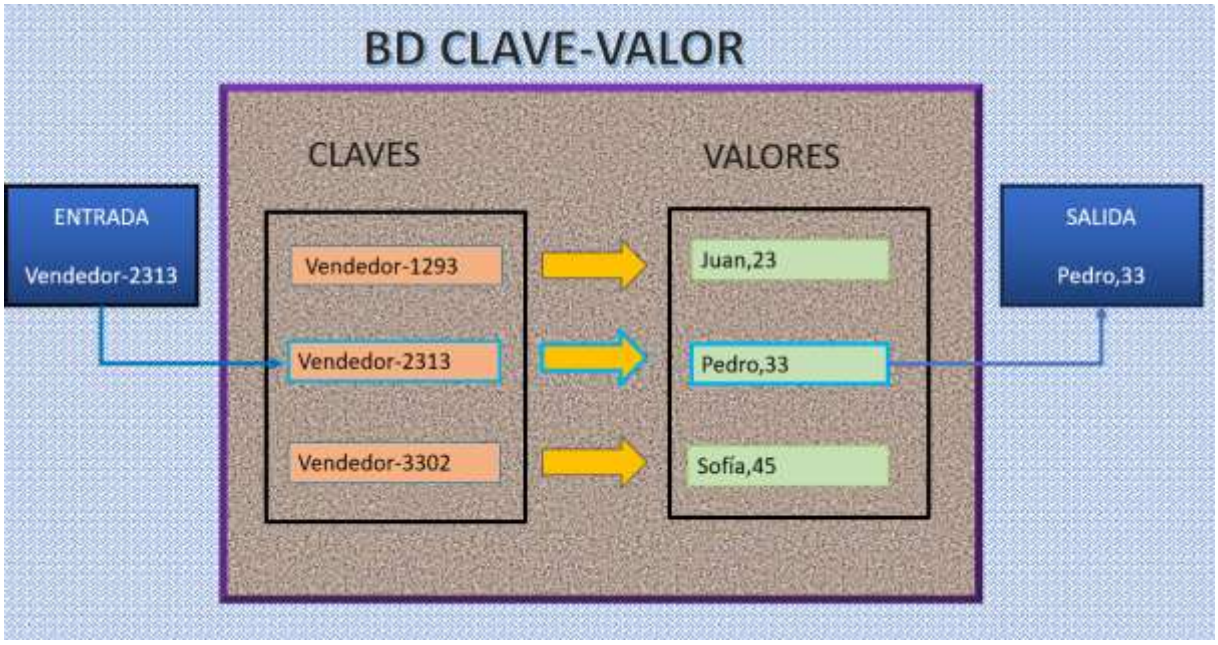


Figura 2. BD clave-valor

2.1.2 BD Orientadas a Documentos

Los desarrolladores suelen recurrir a bases de datos de documentos cuando necesitan la flexibilidad de las bases de datos NoSQL, pero necesitan gestionar estructuras de datos más complejas en comparación con las BD clave-valor.

Este tipo de base de datos, utilizan documentos para el almacenamiento de todos los registros y sus datos asociados. Estos documentos pueden tener diferentes formatos, desde archivos JSON o XML hasta documentos de texto. Cada uno de estos documentos contiene registros puede almacenar distintos tipos de datos.

Se puede asignar un identificador único a cada documento de forma automática, para poder consultar los datos que deseemos.

Estas bases de datos se pueden representar con productos, siendo el más conocido MongoDB, utilizándose para la analítica de grandes cantidades de datos no estructurados, o en aplicaciones móviles para la sincronización de datos.

En la Figura 3 se muestra un ejemplo de este tipo de base de datos, donde tenemos la información estructurada en un documento, en el cual podemos consultar los datos que deseemos.

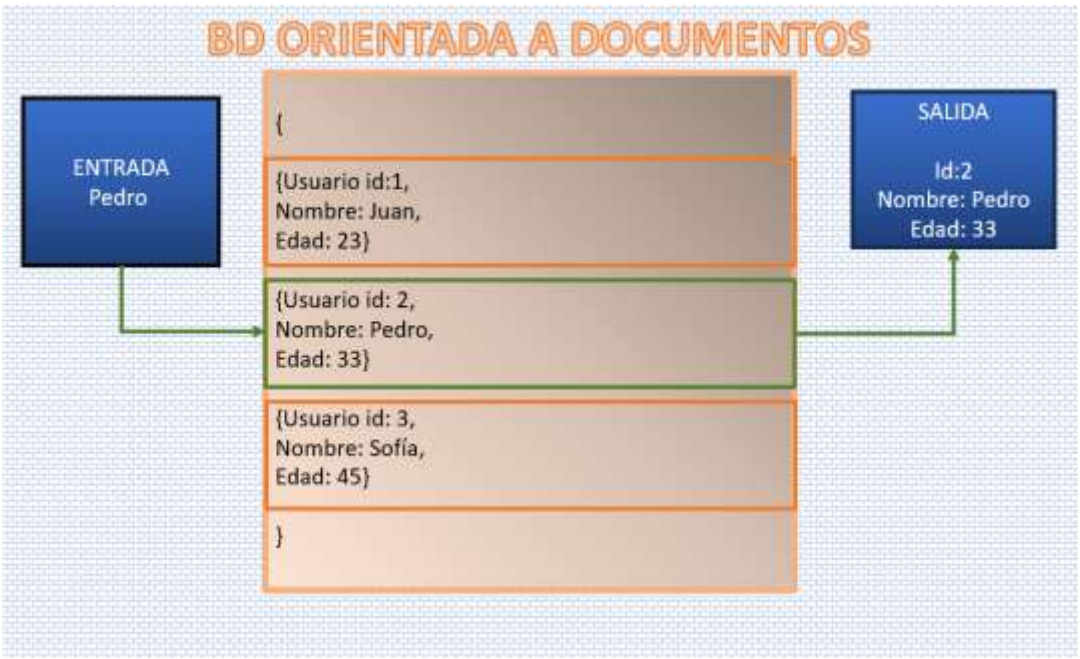


Figura 3 BD Orientada a documentos

2.1.3 BD Familia de Columnas

Este tipo de base de datos NoSQL es muy importante en inteligencia empresarial o inteligencia de negocios y se emplea en empresas donde posteriormente los datos pueden ser utilizados para optimizar la toma de decisiones y aumentar el rendimiento de la organización.

Los datos se almacenan en columnas en lugar de filas. Esto permite que se lean y procesen solo las columnas necesarias para una consulta específica, lo que puede mejorar el rendimiento cuando solo se necesitan unas pocas columnas de un conjunto de datos grande.

En resumen, las bases de datos orientadas a columnas son una excelente opción para aplicaciones que requieren un rendimiento eficiente en consultas de lectura, análisis de datos y procesamiento de grandes volúmenes de información, sobre todo en entornos distribuidos.

Para este tipo de base de datos se usan productos, como Apache Cassandra.

En la Figura 4 se muestra un ejemplo de este tipo de base de datos. Como podemos observar los datos de cada persona aparecen agrupados en una misma columna, a la hora de hacer la consulta de Pedro se obtienen todos los datos de la columna.



Figura 4 BD Familia de Columnas

2.1.4 BD Orientadas a Grafos

Como se indica en [6] las bases de datos orientadas a grafos son herramientas poderosas y especializadas para manejar datos complejos y altamente interconectados, ofreciendo eficiencia y rendimiento en comparación con otros tipos de bases de datos.

En estas bases de datos, la estructura de datos fundamental es el grafo, compuesto por nodos, aristas y propiedades. Cada nodo puede tener propiedades (atributos) que describen características de la entidad. Las aristas representan las relaciones entre los nodos, puede tener también propiedades y una dirección (si es un grafo dirigido).

Como se indica en [7] cabe destacar que en una base de datos orientada a grafos los nodos pueden estar etiquetados, es decir, cada nodo puede poseer una etiqueta que tiene como función clasificarlo en el grafo. En el caso de las aristas ocurre lo mismo, deben contener una etiqueta (o tipo).

Este tipo de bases de datos es particularmente útil para gestionar y consultar datos con relaciones complejas e interconectadas, como puede ser el uso de aplicaciones de redes sociales para conectar relaciones entre usuarios y la información de cada uno.

Uno de los productos más destacados de gestión de BD orientadas a grafos es Neo4J.

En este TFG nos centraremos en este tipo de bases datos. En la Figura 5 se muestra un ejemplo, en el cual existen nodos con distintas etiquetas diferenciados por el color (azul para actividades y marrón para personas) y las relaciones entre los nodos representadas por las aristas que también van diferenciadas por su etiqueta (color rojo corresponde a una relación de amistad, color azul corresponde a una relación de practica).



Figura 5 BD Orientada a grafos

2.2 Neo4J

Neo4J es un gestor de bases de datos orientadas a grafos que utiliza la teoría de grafos para almacenar y gestionar datos. Utiliza nodos, relaciones (aristas) y propiedades para representar y almacenar información.

Esto permite un modelado natural, siendo este producto puntero en el mercado debido a su capacidad de manejar estructuras de datos complejas e interconectadas. Otra de sus principales características es el poder de trabajar con un gran volumen de datos, en especial datos que implican múltiples niveles de relaciones.

Otra de sus ventajas es la seguridad ya que permite definir permisos detallados para distintos usuarios y roles.

Neo4J obtiene las ventajas y características de las bases de datos orientadas a grafos, el manejo de múltiples datos y sus relaciones, siendo muy útiles en el uso de las redes sociales. Además, Neo4J cuenta con una buena variedad de algoritmos de grafos, como, por ejemplo, la búsqueda de caminos más cortos, detección de comunidades, análisis de centralidad, etc.

En resumen, Neo4j es una herramienta poderosa y flexible para cualquier aplicación que requiera manejar y analizar datos altamente conectados, proporcionando un conjunto robusto de características que facilitan su implementación y uso en diversos contextos industriales y académicos.

En este TFG vamos a utilizar Neo4J para crear la BD orientada a grafos tras aplicar las guías de transformación propuestas a un caso de estudio. En Neo4J vamos a crear la base de datos orientada a grafos y realizar consultas para validar las guías.

Además, como se indica en [8] Neo4J utiliza Cypher como lenguaje de programación tanto para crear la base de datos como para hacer consultas. Cypher es un lenguaje de consulta declarativo y expresivo diseñado específicamente para trabajar con grafos. Permite crear, leer, actualizar y eliminar nodos y relaciones de manera intuitiva y eficiente.

III. DESARROLLO

El objetivo principal de este capítulo es definir unas guías de transformación para que, partiendo de un esquema conceptual, lleguemos a la obtención de una base de datos orientada a grafos.

Para ello, en primer lugar, se ha realizado el caso de estudio, que recoge diferentes tipos de elementos del Modelo Entidad/Interrelación. Iremos descomponiendo el esquema conceptual del caso de estudio para abordar la especificación de las distintas reglas.

La idea es evaluar todas las opciones existentes para definir las guías más adecuadas que lleven a la obtención de la base de datos orientada a grafos, intentando siempre recoger la mayor cantidad de semántica posible.

3.1 Caso de Estudio

Pasamos a definir el caso de estudio, a partir del cual vamos a desarrollar las guías de transformación y su validación.

Como se observa en la Figura 6, el esquema conceptual del que partiremos presenta la siguiente situación: Un empleado ocupa de forma obligatoria una única plaza de trabajo en una universidad. En el ejemplo tenemos una jerarquía donde apreciamos tres subtipos distintos ligados con el empleado: profesor, becario y personal de mantenimiento. A su vez, estos tres subtipos mantienen las siguientes relaciones:

- Profesor: un profesor debe impartir obligatoriamente una o más asignaturas.
- Becario: un becario puede tener de uno a varios compañeros becarios en su trabajo.
- Personal de mantenimiento: un trabajador de mantenimiento puede realizar su trabajo en uno o más campus universitario. Mientras que en un mismo campus pueden trabajar todos los trabajadores que se necesiten.

En la siguiente figura podemos ver el esquema conceptual que nos servirá como base para definir las guías de transformación.

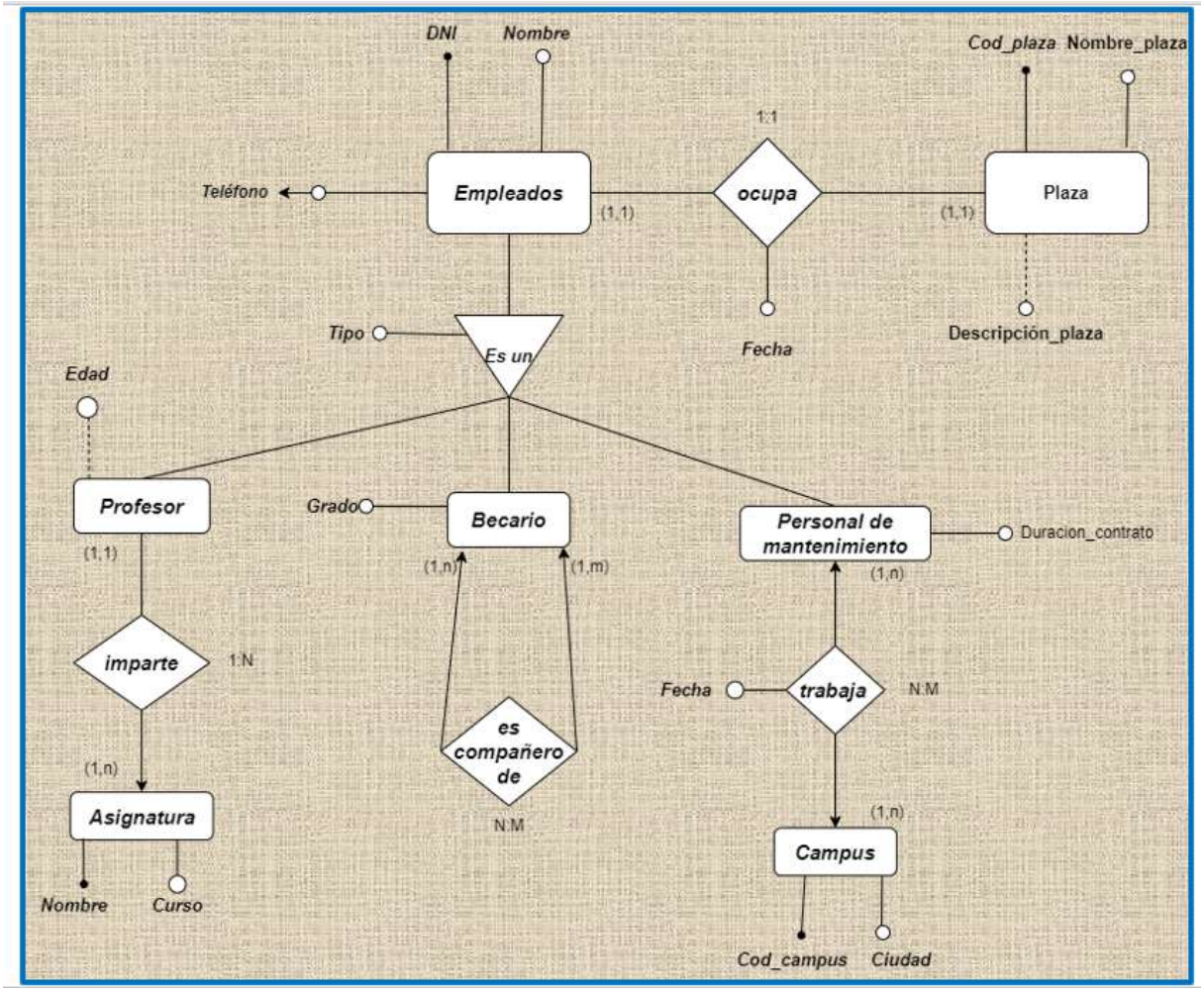


Figura 6 Esquema conceptual del caso de estudio.

3.2 Diseño de BD orientada a Grafos

Establecemos una serie de guías básicas para el diseño de la base de datos orientada a grafos a partir de un esquema conceptual.

3.2.1 Transformación de Entidades y Atributos.

A la hora de transformar las entidades haremos la siguiente transformación:

- Ocurrencia de la entidad del modelo E/R ⇔ Nodo del grafo.
- Nombre de la entidad del modelo E/R ⇔ Etiqueta del nodo del grafo.

Acorde a los puntos anteriores, comenzamos por transformar la entidad en un nodo del grafo. El nombre de la entidad de nuestro esquema conceptual, equivale al nombre de la etiqueta de nuestro nodo.

Por ejemplo, en nuestro caso de estudio, vamos a escoger la entidad 'asignatura' y le aplicamos la guía propuesta para ver su diseño en un grafo. La entidad asignatura se transforma en un nodo del grafo, teniendo una etiqueta denominada asignatura.

En la Figura 7 vemos la representación de la transformación de la entidad asignatura después de aplicarle las guías, como se observa es un nodo con su etiqueta correspondiente.



Figura 7 Transformación de la entidad asignatura

A la hora de transformar los atributos de las entidades estos se transforman como atributos de su nodo correspondiente independientemente de su tipo en la entidad del modelo E/R.

Observamos como quedaría en el caso de la entidad 'asignatura', aplicando la guía los atributos de la entidad (nombre y curso) son atributos del nodo con etiqueta asignatura del grafo.

En la Figura 8 observamos como queda el nodo tras la transformación.



Figura 8 Transformación atributos del modelo E/R

A la hora de llevarnos los atributos de la entidad al nodo, debemos considerar los distintos tipos de atributo. Aunque todos los atributos de la entidad acaben siendo atributos del nodo, es necesario aplicar guías para mantener el tipo de atributo en el nodo. Exponemos a continuación guías para transformar los distintos tipos de atributos:

- Atributo único de la entidad ⇔ Restricción atributo único del nodo.

A un atributo único de la entidad, imponemos como restricción la obligatoriedad de que sea también único en el nodo. En el caso de la entidad 'Asignatura' tenemos el atributo 'nombre' que debe ser único. Para que este atributo sea único en el nodo, como reflejamos en la Figura 9, le aplicamos la restricción en Neo4J, para que sea único. Con ello no nos dejará insertar asignaturas con el mismo nombre.

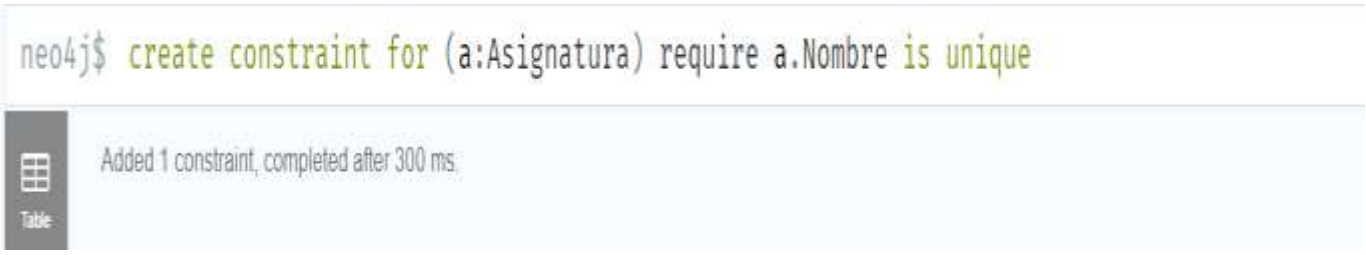


Figura 9 Restricción atributo único.

- Atributo obligatorio de la entidad ⇔ Restricción atributo obligatorio del nodo.

En un atributo obligatorio de la entidad, definimos una restricción que obligue a que exista un valor para ese atributo. Antes de que el atributo de la entidad se convierta en atributo del nodo, debemos aplicarle la restricción.

En la Figura 10 se aprecia la restricción aplicada a modo de ejemplo del atributo 'curso' de la entidad asignatura, este atributo debe tener siempre valor, por ello establecemos la siguiente restricción, por la cual nunca puede ser nulo.

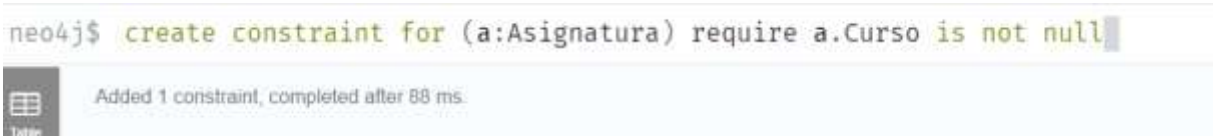


Figura 10 Restricción atributo no nulo

- Atributo multivaluado de la entidad ⇔ Atributo multivaluado del nodo.

Entendemos como atributo multivaluado, un tipo de atributo que admite varios valores dentro del mismo, por tanto, tendríamos un array de valores como atributo dentro del nodo. A modo de ejemplo el atributo 'teléfono' de la entidad 'empleado', sería capaz de admitir el número de teléfono fijo, el número de teléfono móvil e incluso otros teléfonos dentro del mismo atributo 'teléfono', sin necesidad de crear un atributo para cada uno de ellos.

- Atributo opcional de la entidad ⇔ Atributo opcional del nodo

Los atributos opcionales de la entidad, pueden existir o no en el nodo. Al crear una instancia de un nodo, si no tenemos el atributo opcional, este no existiría en el nodo. Si, por el contrario, queremos dar valor al atributo opcional, se convertirá en atributo del nodo. Imaginemos que en una asignatura puede haber un libro de ayuda, se guardará en un atributo opcional. Al crear el nodo si la asignatura no tiene el libro, el atributo no aparecerá en el nodo.

- Atributo compuesto de la entidad ⇔ Atributo compuesto del nodo

Para representar un atributo compuesto en un nodo, guardamos el atributo en el nodo, y dentro podemos tener los distintos subtipos con sus valores. Por ejemplo, imaginemos que en

la entidad asignatura tenemos un atributo compuesto llamado universidad, donde queremos guardar el nombre y la ciudad de la universidad.

En la Figura 11 vemos que en el nodo se define el atributo compuesto 'universidad', dentro del cual guardamos la información de los subtipos.



Figura 11 Representación atributo compuesto

Estas han sido las reglas básicas que iremos utilizando para transformar las entidades y atributos de nuestro esquema conceptual.

A continuación, pasamos a definir las guías relativas a las interrelaciones, diferenciando:

- Interrelación 1:1.
- Interrelación 1:N.
- Interrelación 1:M.
- Interrelación reflexiva.
- Jerarquía.

3.2.2 Transformación Interrelación 1:1

Vamos a transformar una interrelación de tipo 1:1 del modelo conceptual a una base orientada a grafos. Para realizar esta transformación partiremos de la interrelación 'ocupa' con un tipo de correspondencia 1:1.

Como expresa la Figura 12 vamos a partir de esta interrelación de nuestro esquema conceptual, donde un empleado ocupa siempre una sola plaza o puesto de trabajo, y una plaza únicamente está ocupada por una sola plaza.

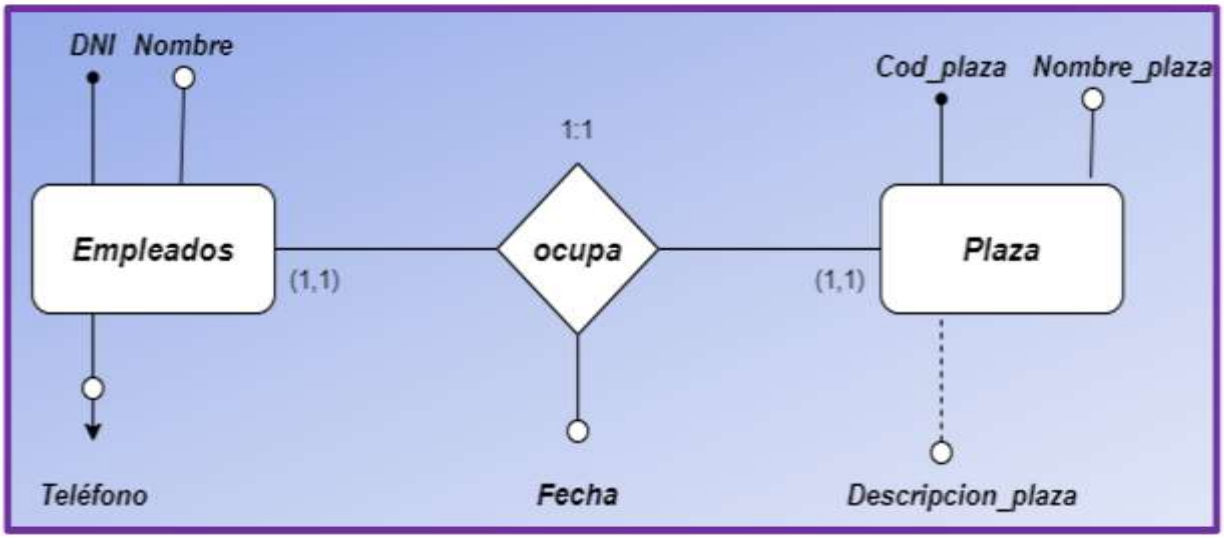


Figura 12 Interrelación 1:1

El primer paso es evaluar las distintas opciones para transformar la interrelación y escoger la más adecuada.

Una de estas opciones podría ser transformar las entidades 'Empleados' y 'Plaza' en nodos, y establecer una arista para transformar la interrelación 'ocupa' para tener ambos nodos relacionados. La ventaja de esta posibilidad es que tenemos los datos bien organizados, y relacionados por aristas lo cual facilita la comprensión. La desventaja es que no podríamos asegurar que un empleado ocupe como mínimo una plaza ya que no disponemos de restricciones a la hora de establecer relaciones, por esta razón no escogeremos esta opción.

Otra de las opciones es transformar la interrelación en un único nodo en el grafo. En nuestro ejemplo tendríamos un nodo en el cual guardaríamos los atributos de las entidades 'Empleados' y 'Plaza', de esta manera en un solo nodo tenemos ambas entidades relacionadas.

La desventaja de esta opción es que todos los atributos de las dos entidades van a recaer en un mismo nodo lo cual no es ideal, ya que guardamos toda la información en un nodo y

puede no ser eficiente. Además de que desaparece el empleado y la plaza, ahora solo existirá un nodo representado ambas.

Las ventajas son que en este caso si podemos asegurar que un empleado solo ocupe una plaza mediante las restricciones de los atributos, esto hace que no perdamos semántica, al crear un empleado va a tener su plaza asociada con toda la información sin la posibilidad de que un empleado ocupe varias plazas o no ocupe ninguna. Otra ventaja es que nos ahorramos crear varios nodos y aristas, de forma que con un solo nodo podemos representar la relación.

Por tanto, como esta última opción mantiene de forma más correcta nuestro esquema conceptual, definamos la guía de transformación paso por paso:

- Interrelación entre entidades del modelo E/R ⇔ Nodo de grafo.

Transformamos la interrelación 'ocupa' en un nodo de nuestro grafo. Etiquetamos el nodo con la etiqueta 'Empleados' al tener interrelación propia.

Hemos conseguido ahorramos representar las entidades 'Empleados' y 'Plaza'. En la Figura 13 vemos como nuestro grafo está compuesto por un único nodo con etiqueta 'Empleados'.



Figura 13 Interrelación 1:1 Nodo

El siguiente paso es saber cómo representar todos los atributos de las entidades empleados y plazas en el nodo 'Empleados', para lo cual realicemos los siguientes puntos:

- Atributos de las entidades ⇔ Atributos del nodo que representa la relación.
- Atributos de la interrelación ⇔ Atributos del nodo que representa la relación.

El nodo creado con etiqueta ‘Empleados’ será el encargado de relacionar a las entidades empleado y plaza. La guía propuesta es que dicho nodo tenga todos los atributos de ambas entidades, en este caso, empleado y plaza; manteniendo los tipos de los atributos de la misma manera que están en sus entidades como vimos para cada atributo en las guías básicas.

Hay que tener en cuenta que la interrelación ampara un atributo ‘Fecha’, que deberemos meter en el nodo ‘Empleados’.

Aplicando estas guías, en la Figura 14 se muestra cómo queda esta parte en el grafo. En él tenemos un solo nodo ‘Empleados’ donde almacenamos toda la información del empleado y su respectiva plaza.



Figura 14 Interrelación 1:1 Atributos

Cabe destacar que, según la cardinalidad de la relación, puede haber cambios a la hora de establecer las relaciones y atributos. En nuestro ejemplo, como un empleado debe ocupar obligatoriamente una plaza, en el nuevo nodo hemos mantenido que el DNI del empleado y el cod_plaza sean únicos y obligatorios.

Imaginemos que un empleado no necesariamente deba ocupar una plaza, es decir cardinalidad (0,1), en el nodo ‘ocupa’ que hemos creado con la guía, deberíamos indicar que la plaza debe ser única pero no haría falta la restricción de ser nula.

En resumen, hemos partido de un modelo conceptual con una interrelación entre dos entidades y aplicando nuestras guías hemos obtenido un grafo que contiene un nodo con una etiqueta que representa la relación entre las dos entidades y contiene los atributos de ambas.

3.2.3 Transformación Interrelación 1:N

En este apartado definiremos una serie de reglas para transformar una interrelación 1:N de nuestro esquema conceptual hasta llegar a una base de datos orientada a grafo. Para desarrollar las guías de transformación partimos de nuestro esquema del caso de estudio, más en concreto nos centraremos en la interrelación 1:N, 'imparte'.

Como observamos en la Figura 15, para relacionar profesores con asignaturas podemos decir que un profesor puede dar clases en una o varias asignaturas. En nuestro ejemplo, una asignatura solo la imparte un profesor (1,1).

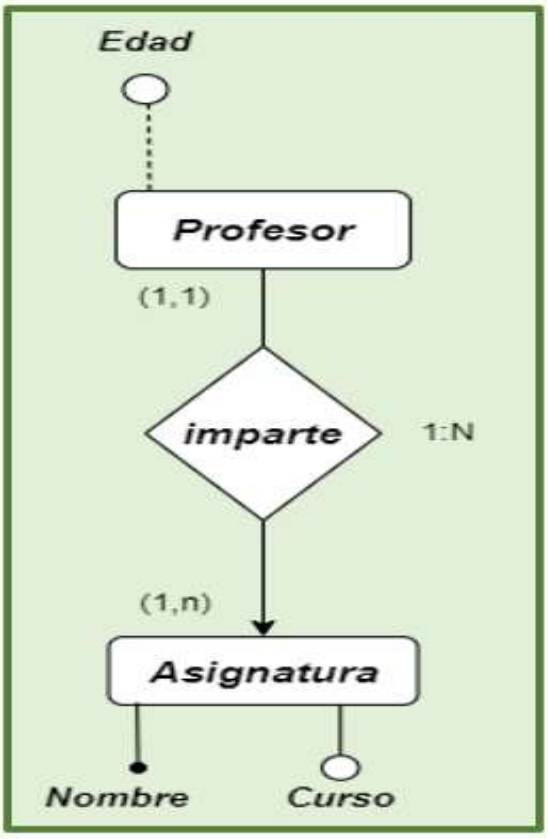


Figura 15 Interrelación 1:N

Empezamos planteando las distintas alternativas para transformar la interrelación. Una de las opciones es como hicimos en la anterior interrelación, representarlo todo en un solo nodo, dentro del nodo guardaríamos la información tanto del profesor como de las asignaturas que imparte. Con esto mantenemos la misma ventaja, ya que sería sencillo representar la ocurrencia

de que una asignatura solo sea impartida por un profesor aplicando la restricción de unicidad a la asignatura.

Sin embargo, esta opción no la consideramos óptima para este tipo de interrelación, ya que un profesor se relaciona con n asignaturas y tendríamos la información de la asignatura embebida en el profesor.

Desechado lo anterior, desarrollamos como idea convertir las entidades en nodos del grafo y establecer la interrelación entre ellas mediante aristas. De esta forma solo tendríamos una instancia 'Profesor' relacionada mediante aristas que apuntan a sus asignaturas. Esta forma es más fácil de desarrollar, dejando la información de forma más recogida.

En cuanto a la semántica conseguimos mantener todos los datos de los profesores y asignaturas, sin embargo, no podemos asegurar que un profesor imparta como mínimo una asignatura, entonces en nuestro grafo podemos tener profesores sin impartir asignaturas.

Comenzamos con las guías de transformación paso por paso hasta llegar a la base de datos orientada a grafos:

En primer lugar, transformamos las entidades y sus atributos tal y como aparece en la Figura 16, en la que tenemos un nodo 'Profesor' y un nodo 'Asignatura' con sus respectivos atributos.

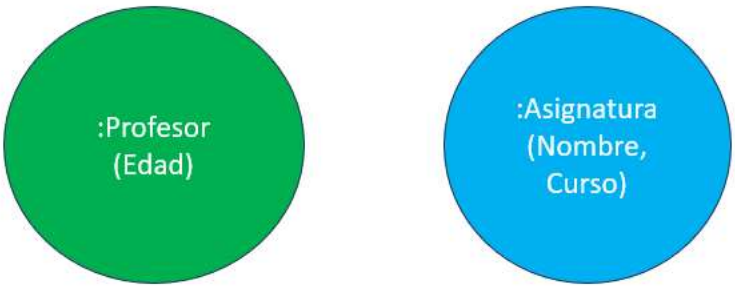


Figura 16 Interrelación 1:N con nodos

Como ya tenemos los nodos que representan las entidades del esquema conceptual, ahora vamos a definir las reglas para la transformación de la interrelación.

- Interrelación del esquema ⇔ Arista en grafo

- Nombre de la interrelación ⇔ Tipo (etiqueta) de la arista del grafo

Para poder establecer la interrelación del esquema conceptual, vamos a crear una arista que relacione a los nodos del grafo (profesor y asignatura), desde el nodo profesor hasta el nodo asignatura.

Como la interrelación es de cardinalidad (1,n), desde un nodo 'profesor' vamos a crear 'n' aristas dirigidas a las instancias u ocurrencias de 'Asignatura' que imparta.

Al igual que los nodos, las aristas de un grafo van etiquetadas. Transformamos en nombre de la interrelación del esquema conceptual en el nombre de la etiqueta de la arista.

Las aristas creadas desde el nodo profesor al nodo asignatura, llevarán como etiqueta 'imparte'.

En la Figura 17 apreciamos el resultado de la guía de transformación, en la cual tenemos un nodo profesor con aristas dirigidas a instancias de asignaturas.

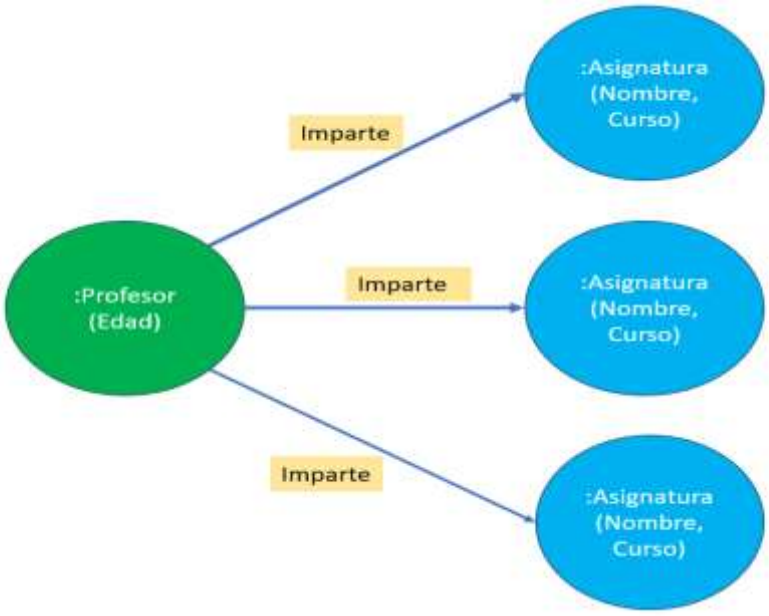


Figura 17 Interrelación 1:N aristas

Es muy importante a la hora de crear una arista ver su dirección, debiendo llevar la misma dirección con que se representa en el esquema conceptual y teniendo en cuenta las consultas más frecuentes, en una base datos orientada a grafos no es necesario crear aristas desde los nodos asignatura al nodo profesor, a la hora de hacer consultas en Neo4j, la asignatura

queda relacionada con el profesor mediante la arista 'imparte' (la relación se podría recorrer en los dos sentidos).

3.2.3 Transformación Interrelación N:M

En este apartado definiremos las guías para transformar una interrelación N:M.

Como se ve en la imagen de la Figura 18 queremos relacionar al personal de mantenimiento con el campus en el que trabaja. Un empleado de mantenimiento trabaja en N campus.

También hay que representar que en un campus trabajan M empleados de mantenimiento.

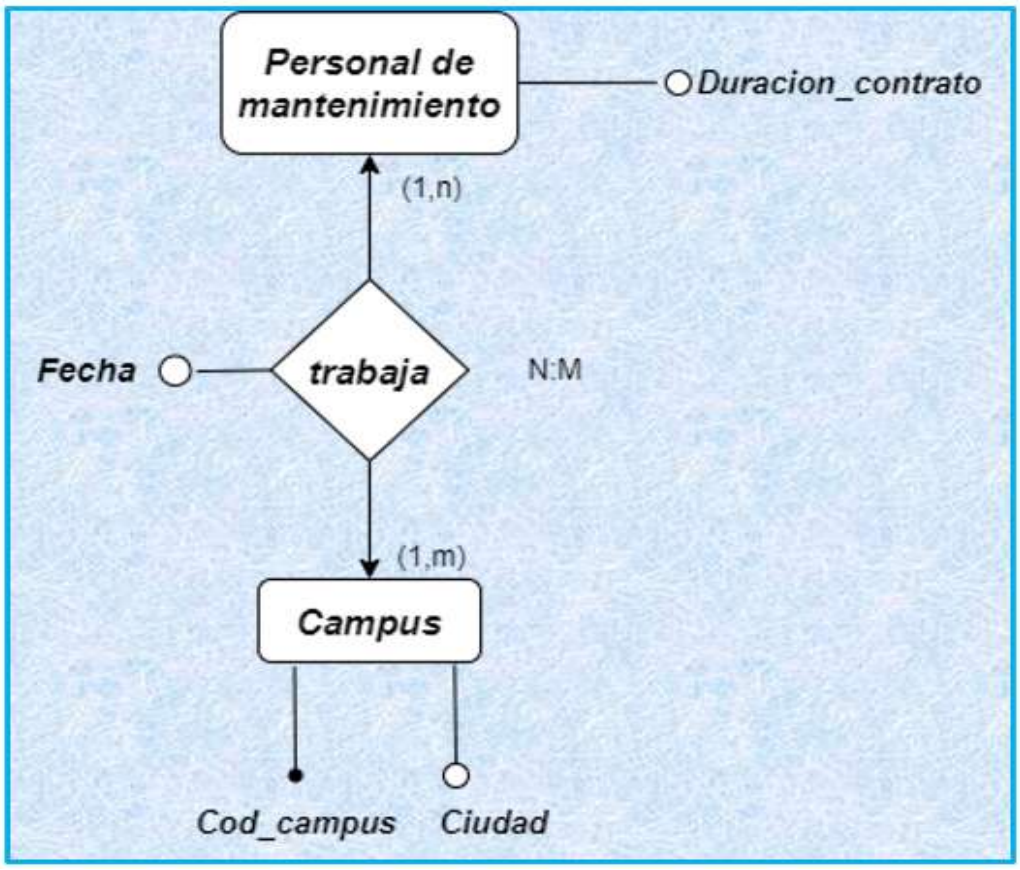


Figura 18 Interrelación N:M

De nuevo, comenzamos estudiado las posibilidades para poder transformar este tipo de interrelación.

En este caso la idea de crear un solo nodo para representar la interrelación, no la consideramos ideal, debido a que necesitamos demasiadas instancias tanto para representar la entidad 'Personal de mantenimiento' como para la entidad 'Campus', lo cual hace esta idea aún más laboriosa que en el resto de interrelaciones, por su cantidad de relaciones.

Por tanto, la opción que consideramos más viable es representar las entidades como nodos y crear aristas para relacionar esos nodos. Es una idea similar a la expuesta en la interrelación 1:N pero con matices, derivados de que tenemos que un 'Personal de mantenimiento' trabaja en 'n' 'Campus', pero a su vez en un campus trabajan 'n' 'Personal de mantenimiento'.

Las guías para la transformación de la interrelación son:

En primer lugar, transformamos las entidades y los atributos como se observa en la Figura 19. El resultado son dos nodos, uno por cada entidad, con su etiqueta y atributo correspondiente.



Figura 19 Interrelación N:M Nodos

Ahora definimos los pasos para transformar la interrelación 'trabaja' del esquema conceptual:

- Interrelación entre entidades ⇔Arista del grafo
- Nombre de la interrelación ⇔ Etiqueta de la arista del grafo

La idea es crear aristas que relacionen los nodos del grafo (personal de mantenimiento y campus).

Primeramente, creamos las aristas desde el nodo 'Personal de mantenimiento' a todas las instancias de 'campus' que tengamos.

La interrelación N:M obliga a relacionar un ‘Campus’ con sus ‘n’ ‘Personal de mantenimiento’.

En Neo4J no existe una arista bidireccional, por esta razón de la misma forma en que hemos creado aristas desde el nodo de ‘personal de mantenimiento’ apuntando a las instancias ‘Campus’, creamos una arista desde el nodo campus apuntando a todas las instancias del nodo ‘Personal de mantenimiento’.

A la hora de crear la arista debemos darle una etiqueta. Transformamos el nombre de la interrelación del esquema conceptual en el nombre de la etiqueta de la arista.

Las aristas creadas desde el nodo ‘Personal de mantenimiento’ al nodo ‘Campus’, llevarán como etiqueta ‘trabaja’.

En la Figura 20, vemos la representación de estas guías, en la cual vamos a tener varias instancias de los nodos ‘Personal de mantenimiento’ y ‘Campus’. Observamos como una instancia de personal de mantenimiento se relaciona con una arista ‘trabaja’ con los campus en los que trabaja. A su vez desde un nodo campus también salen aristas con destino las instancias de ‘Personal de mantenimiento’ que trabajan en él.

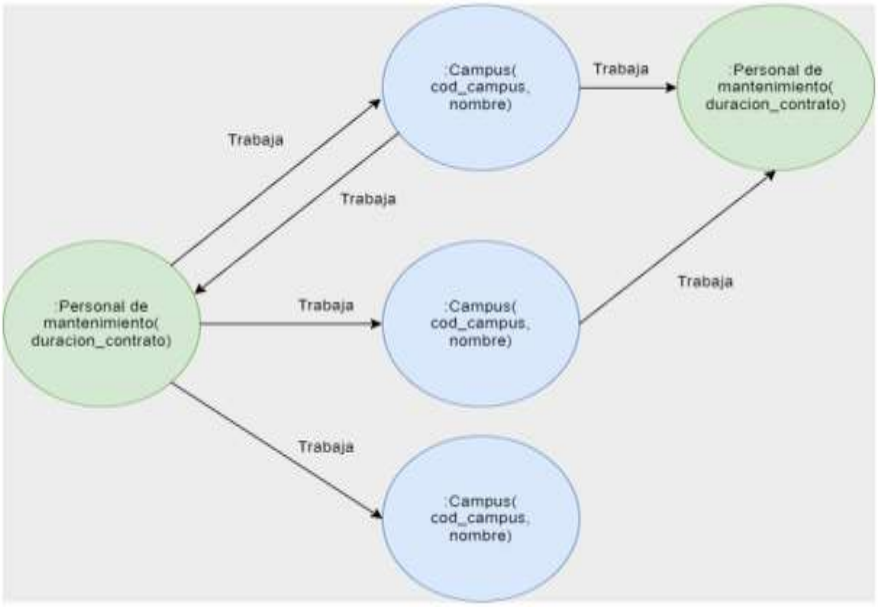


Figura 20 Interrelación N:M Aristas

Solo falta llevar a nuestro grafo el atributo ‘fecha’ de la interrelación, para ello:

- Atributo de la interrelación ⇔ Atributo de la arista del grafo

Habría que representar el atributo 'fecha' de la interrelación para convertirlo en un atributo de la arista con etiqueta 'trabaja'.

A las aristas del grafo les añadimos el atributo 'fecha'. El resultado se muestra en la Figura 21.

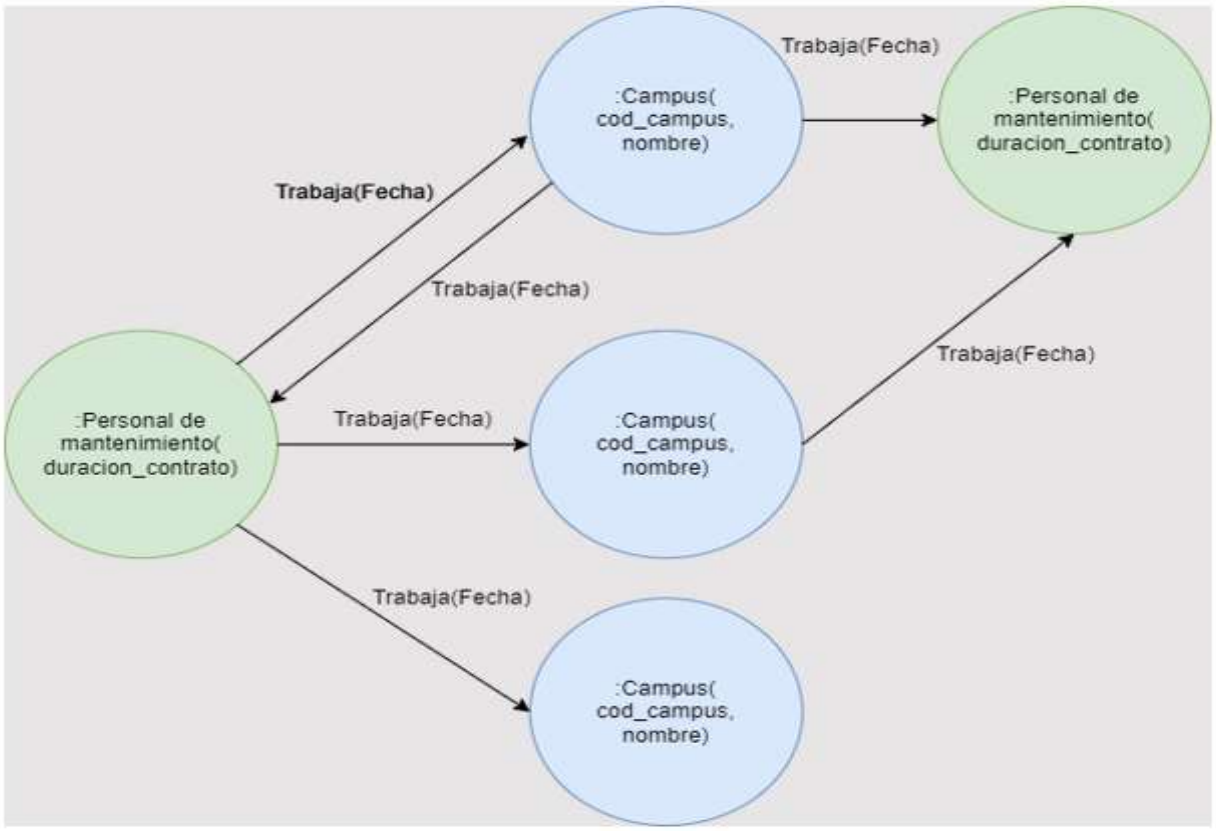


Figura 21 Interrelación N:M Aristas atributo

3.2.4 Transformación Interrelación reflexiva

Una interrelación reflexiva es aquella donde la entidad se relaciona consigo misma.

En este apartado volvemos a definir las guías de transformación de una interrelación reflexiva.

En la Figura 22 apreciamos como la entidad becarios se puede relacionar consigo mismo 'n' veces. Un becario puede tener 'n' compañeros de equipo, o lo que es lo mismo 'n' becarios.

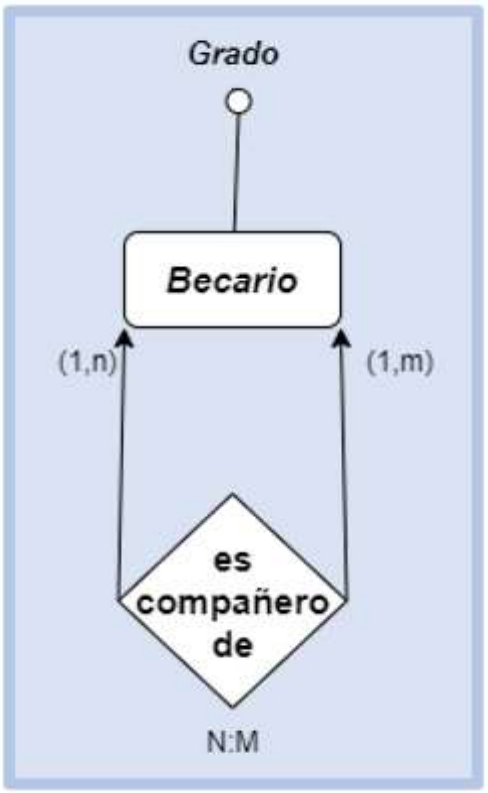


Figura 22. Interrelación N:M reflexiva

Debe tenerse en cuenta que, en las bases de datos orientadas a grafos, se pueden establecer aristas entre nodos con la misma etiqueta. Esta característica hace que para las interrelaciones reflexivas podamos hacer la transformación de la misma forma que hemos hecho en su tipo de interrelación.

Por tanto, como las guías propuestas van a ser las mismas que las guías propuestas en la interrelación N:M, nombraremos las guías para ir mostrando paso por paso como queda en la interrelación reflexiva.

La guía de transformación es la siguiente:

En primer lugar, transformamos la entidad y sus atributos, el resultado se muestra en la Figura 23.



Figura 23. Interrelación N:M Nodo reflexiva

En segundo lugar, transformamos la interrelación:

- Interrelación entre entidades ⇔Arista del grafo
- Nombre de la interrelación ⇔ Etiqueta de la arista del grafo

Se pueden crear aristas entre nodos con la misma etiqueta. Por ello, de forma directa creamos arista que parte desde el nodo 'Becario' y como destino tiene también el propio nodo 'Becario'.

En la figura 24 vemos como queda en el grafo la aplicación de las guías.

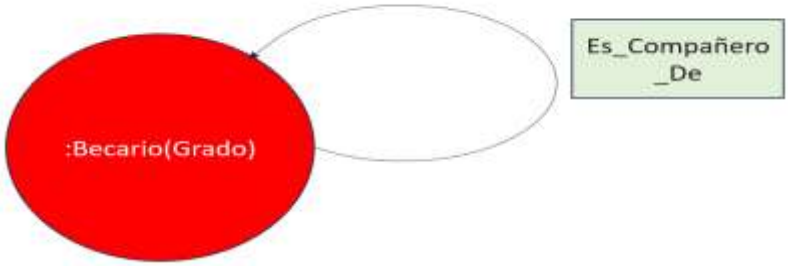


Figura 24. Interrelación N:M Aristas

3.2.5 Transformación Jerarquías

En este punto vamos a hablar de cómo transformar una jerarquía de nuestro esquema conceptual en una base de datos orientada a grafos. Antes de meternos en las guías de transformación de una jerarquía del modelo E/R, vamos a definir en que consiste este concepto.

Entendemos por jerarquía una relación de clasificación entre entidades, donde una entidad más general se subdivide en entidades más específicas. Esta relación se expresa

típicamente mediante una estructura de árbol, donde cada nivel de la jerarquía representa una mayor especialización o detalle.

En una jerarquía, una entidad base, también conocida como supertipo, se caracteriza por tener varias subentidades llamadas subtipos. Cada subtipo hereda los atributos y relaciones del supertipo, pudiendo tener sus propios atributos y relaciones específicas.

Como observamos, la imagen de la Figura 25 muestra una jerarquía en la cual tenemos un supertipo llamado empleados. Los empleados a su vez pueden ser profesores, becarios o personal de mantenimiento de la universidad siendo estas tres opciones subtipos del empleado.

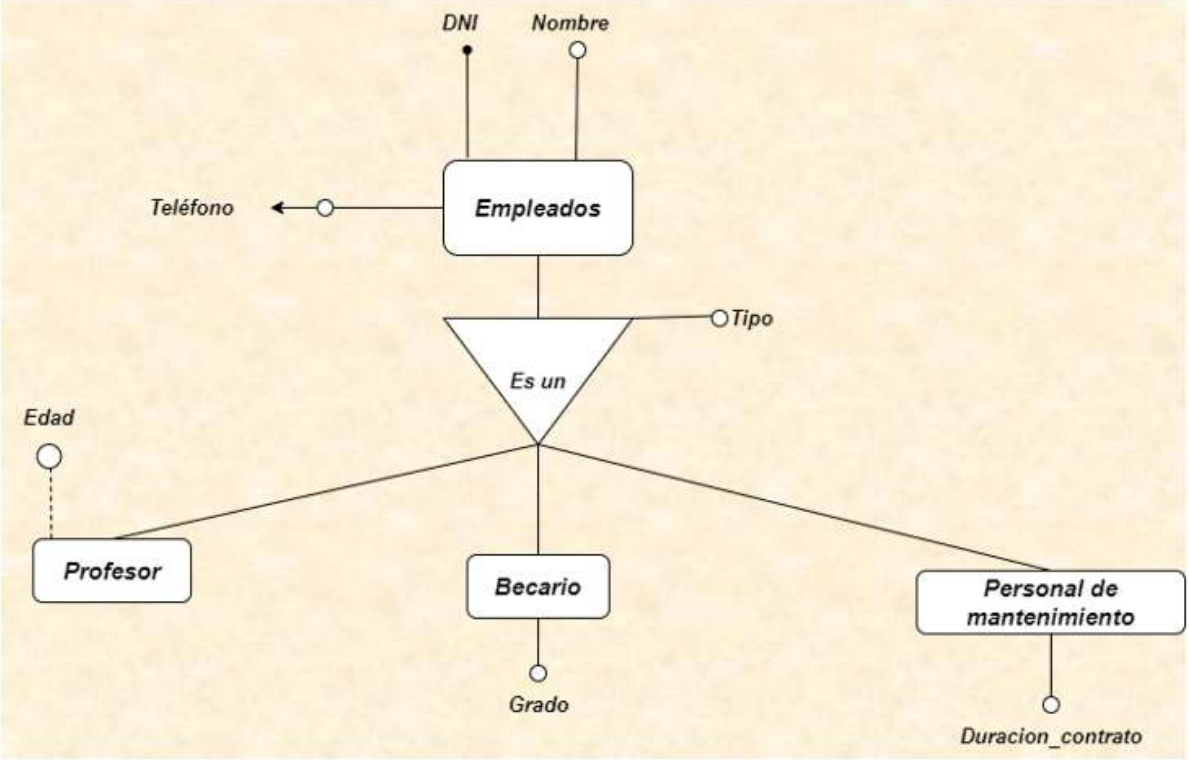


Figura 25 Jerarquía

Existen varios tipos de jerarquías:

- Jerarquía total: los empleados deben pertenecer obligatoriamente a uno de sus subtipos (profesor, becario, personal de mantenimiento).
- Jerarquía parcial: los empleados pueden pertenecer a uno, varios o ninguno de los subtipos (profesor, becario, personal de mantenimiento).

Ambas jerarquías pueden ser solapadas o exclusivas:

- Jerarquías solapadas: entendemos por jerarquía solapada por aquella en donde un empleado puede perteneces a varios subtipos. Por ejemplo, una persona que trabaja en mantenimiento y está en un curso de especialización.
- Jerarquías exclusivas: son aquellas en donde un empleado no puede pertenecer a más de un subtipo.

Acorde a lo antes expuesto, vamos a definir dos guías de transformación distintas, viendo cómo se representa en el grafo y valorando para qué tipo de jerarquía se ajusta más a la guía para perder la menor semántica posible. La primera guía consiste en transformar los subtipos de la jerarquía. En la segunda guía transformaremos simplemente el supertipo para definir la relación. Veámoslo a continuación en detalle:

1) Transformación de los subtipos
   - Subtipo de la jerarquía ⇔ Nodo del grafo
   - Nombre del supertipo ⇔ Primera etiqueta del nodo del grafo
   - Nombre del subtipo correspondiente ⇔ Segunda etiqueta del nodo que corresponde al subtipo.

Una de las ventajas de las bases de datos orientadas a grafos es la posibilidad de que los nodos tengan más de una etiqueta. En base a ello utilizaremos una doble etiqueta convirtiendo en nuestro ejemplo los tres subtipos en nodos con la etiqueta del subtipo y del supertipo. Veámoselo gráficamente.

En la Figura 26 vemos tres nodos representando al subtipo correspondiente, y con doble etiqueta: ‘empleado’ y la etiqueta del subtipo correspondiente.



Figura 26 Jerarquía Nodos

Transformemos ahora los atributos:

- Atributos del supertipo ⇔ Atributos de los nodos que representan a los subtipos
- Atributos del subtipo ⇔ Atributo del nodo de su supertipo

Aplicamos el concepto de heredar. Los atributos del supertipo son heredados por los subtipos, de forma que un subtipo va a contener los atributos de su supertipo. En nuestro ejemplo, todos los nodos subtipo heredarán como atributos, el DNI, nombre y teléfono de la entidad empleado.

Los atributos de la entidad que actúa como subtipo, los vamos transformar como atributos en los nodos que representan al subtipo. En el caso del profesor en el ejemplo, el nodo que representa al profesor contendrá el atributo edad.

En la Figura 27 vemos la representación de la jerarquía en el grafo, en la cual guardamos la relación entre el empleado con sus subtipos.



Figura 27 Jerarquía Atributos

Esta es una de las opciones para transformar la jerarquía que tenemos en el ejemplo del esquema conceptual. Esta forma nos lleva a una jerarquía parcial, debido a que podemos crear empleados de otros tipos, bastaría con crear un nuevo nodo con nueva etiqueta. Con esta opción no podemos tener una jerarquía total.

Por otro lado, esta transformación permite tener a una jerarquía solapada, al definir tres nodos de etiquetas diferentes y no tener ninguna restricción, un empleado puede pertenecer a dos subtipos. Sin embargo, si aplicamos las restricciones de los atributos para que el empleado

sea único podemos conseguir una jerarquía exclusiva al no permitir tener dos instancias del mismo empleado.

2) Transformación del supertipo.
   - Supertipo de la jerarquía ⇔ Nodo del grafo
   - Nombre del supertipo de la jerarquía ⇔ Etiqueta del nodo del grafo

En este caso transformamos el supertipo en un nodo. En nuestro grafo solo vamos a tener un solo nodo 'empleado' donde guardamos la información correspondiente a los subtipos.

Ahora transformamos los atributos de la jerarquía:

- Atributos del supertipo ⇔ Atributos del nodo
- Atributos de los subtipos ⇔ Atributos del nodo
- Atributos de la interrelación ⇔ Atributos del nodo

Acorde a lo anterior, el nodo que representa al supertipo, hereda todos los atributos de la jerarquía.

En nuestro nodo 'Empleados' que corresponde al supertipo guardamos los atributos correspondientes al propio supertipo (DNI, Nombre, Teléfono).

También guardamos el atributo que corresponde a la interrelación (tipo) esto es muy importante, porque en este atributo es donde guardamos el subtipo del empleado y con ello ya tenemos la relación.

Por último, queda guardar los atributos de los subtipos en el nodo, obviamente serán atributos opcionales, cuando tengamos un empleado de tipo profesor le podemos meter el atributo edad, sin embargo, no podemos exigir su obligatoriedad debido a que no siempre será un profesor. En este caso vemos como con esta guía perdemos semántica ya que perderíamos los atributos de los subtipos.

En la figura 28 vemos la representación en el grafo de la jerarquía del esquema conceptual.



Figura 28 Jerarquía atributos supertipo

Analizamos esta representación para saber a qué tipo de jerarquía favorece. En este caso gracias al atributo 'tipo' esta representación nos sirve tanto para jerarquías parciales como totales. Si el atributo 'tipo' no puede ser nulo sería una jerarquía total porque obligamos a pertenecer a uno de los subtipos, mientras que si optamos porque el atributo pueda ser nulo sería una jerarquía parcial.

Esta transformación permite tener una jerarquía exclusiva. Si obligamos a que el empleado sea único, no puede haber más de una instancia de empleado con otros tipos, salvo que el atributo 'tipo' sea multivaluado, en dicho caso al poder guardarnos una lista de valores, podríamos almacenar más de un subtipo dentro del empleado.

3.3 VALIDACIÓN

En este apartado se describe la validación de las guías de diseño de bases de datos orientadas a grafos propuestas. Para ello nos basamos en el modelo conceptual del caso de estudio (Ver Figura 6), aplicamos las guías de transformación propuestas en el capítulo anterior para crear la base de datos en Neo4J, conservando la mayor semántica posible.

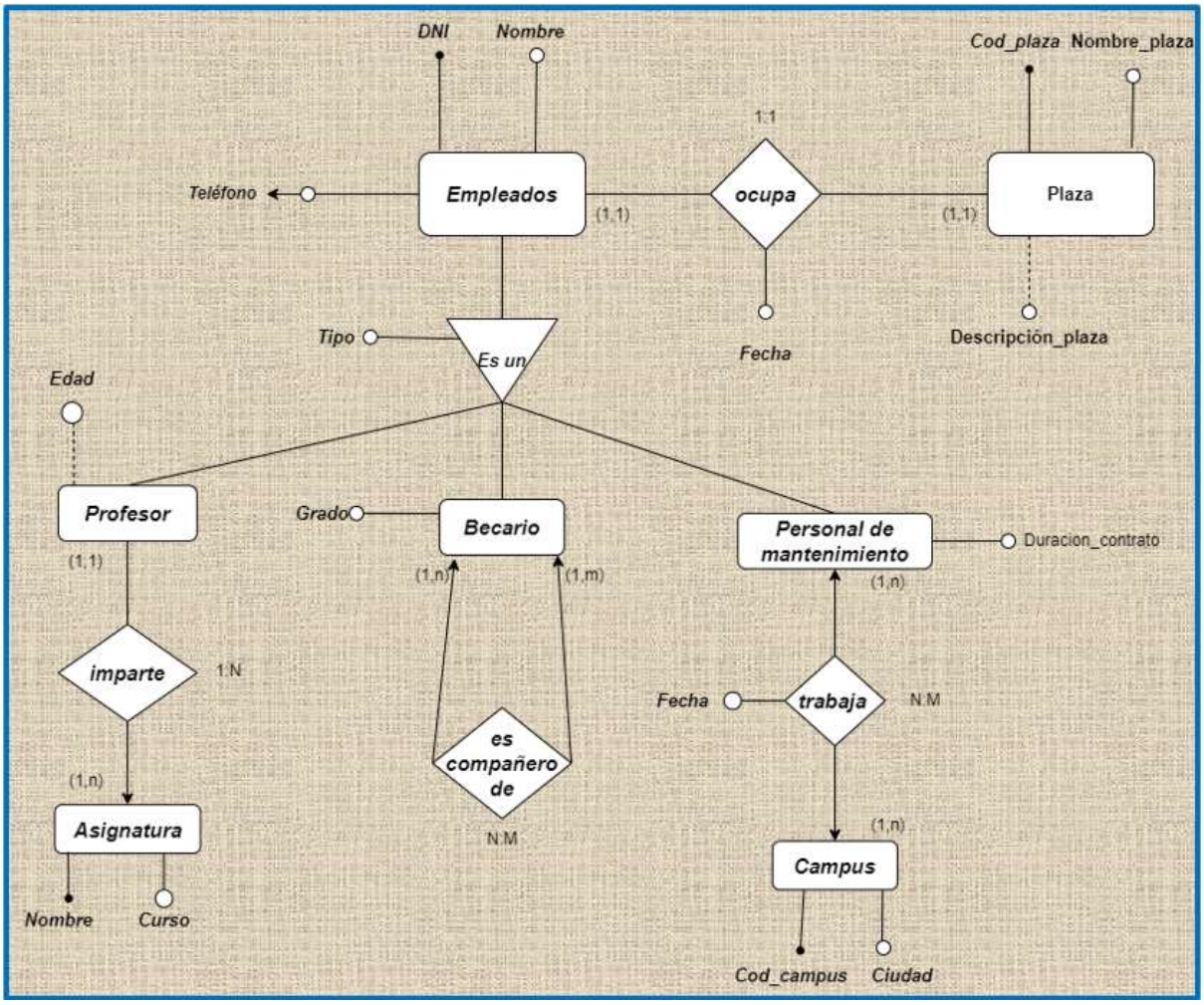


Figura 6 Esquema conceptual del caso de estudio.

A continuación, vamos a describir paso a paso cómo se aplican las reglas, a partir del estudio de las interrelaciones existentes en el esquema conceptual. Después se describe cómo se va realizando la carga usando del lenguaje Cypher.

3.3.1 Transformación de la Interrelación 1:1 - Ocupa

Una vez aplicada la guía adecuada tenemos en la Figura 29 el resultado del esquema conceptual en Neo4J. Vemos como tenemos un nodo empleado, en el cual vamos a guardar toda la información del empleado junto la información de su plaza.



Figura 29 Representación de interrelación 1:1 en Neo4J

Ahora realizamos la carga de un pequeño conjunto de datos de ejemplo. En la figura 30 vemos como queda el grafo en Neo4J después de añadirle un conjunto de empleados, dentro de estos nodos se guarda de manera interna toda la información de la plaza correspondiente.

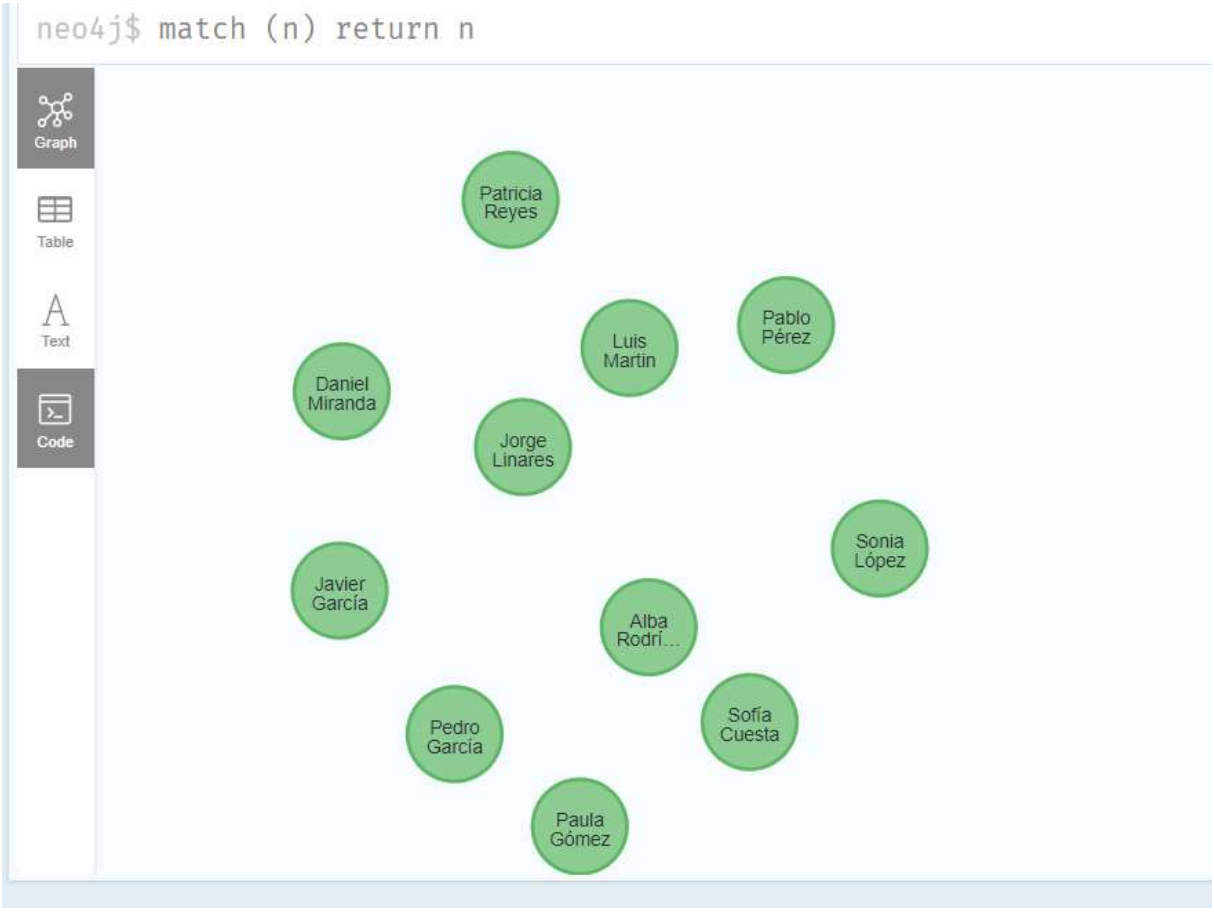


Figura 30 Representación de la Interrelación 1:1 en el grafo de la BD en Neo4J.

Como vemos aplicando la regla de transformación guardamos en un solo nodo toda la información del empleado con su plaza. Vamos a tener once nodos con la misma etiqueta 'Empleado' y en ellos guardamos la información del empleado y de la plaza que ocupa.

Solo queda meter las restricciones correspondientes a los atributos en Neo4J, siendo las vistas en la Figura 31.

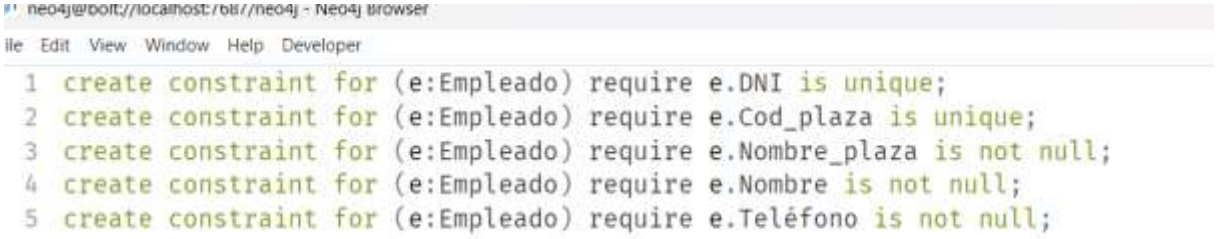


Figura 31 Restricción nodos 'Empleado'

Gracias a estas restricciones podemos ver que no perdemos nada de semántica. Si queremos crear un nuevo empleado que no esté asociado a ninguna plaza, como estamos obligando a que el 'cod_plaza' sea único y el nombre de la plaza no pueda ser nulo, todos nuestros empleados van a estar asociados a una plaza de forma obligatoria.

Para demostrarlo en la Figura 32 se intenta insertar un empleado sin plaza asociada y Neo4J nos muestra el mensaje de error.



Figura 32 Restricción nodos 'Empleado'

Además, estamos manteniendo todos los atributos del empleado y de la plaza, por tanto, con esta guía conseguimos mantener la semántica recogida en el esquema E/R

3.2.2 Transformación de la Jerarquía de Empleados

En esta ocasión vamos a representar la base de datos orientada a grafos una vez aplicadas las guías propuestas para transformar las jerarquías. De las opciones planteadas nos quedamos con la guía que transforma los subtipos en nodos. Con el concepto de la doble etiqueta podemos evitar tener un nodo 'Empleado' como teníamos, ahora tendremos un nodo con doble etiqueta para distinguir el tipo de empleado, heredando en dicho nodo los atributos del empleado y la plaza.

Al aplicar esta guía obtenemos el esquema conceptual de la Figura 33 en Neo4J. Observamos como tenemos los tres subtipos cada uno con su respectivo color diferenciando su tipo. En Neo4J al usar la doble etiqueta también tenemos en el esquema conceptual el nodo 'Empleado'.



Figura 33 Representación Jerarquía en Neo4J

Utilizando los once empleados vistos anteriormente, añadimos la etiqueta correspondiente en Neo4J para diferenciar a los empleados por su tipo (Profesor, Becario, Personal de mantenimiento). En la Figura 34 vemos como tenemos instancias de profesor, becario y personal de mantenimiento, con color azul, rojo y verde respectivamente.

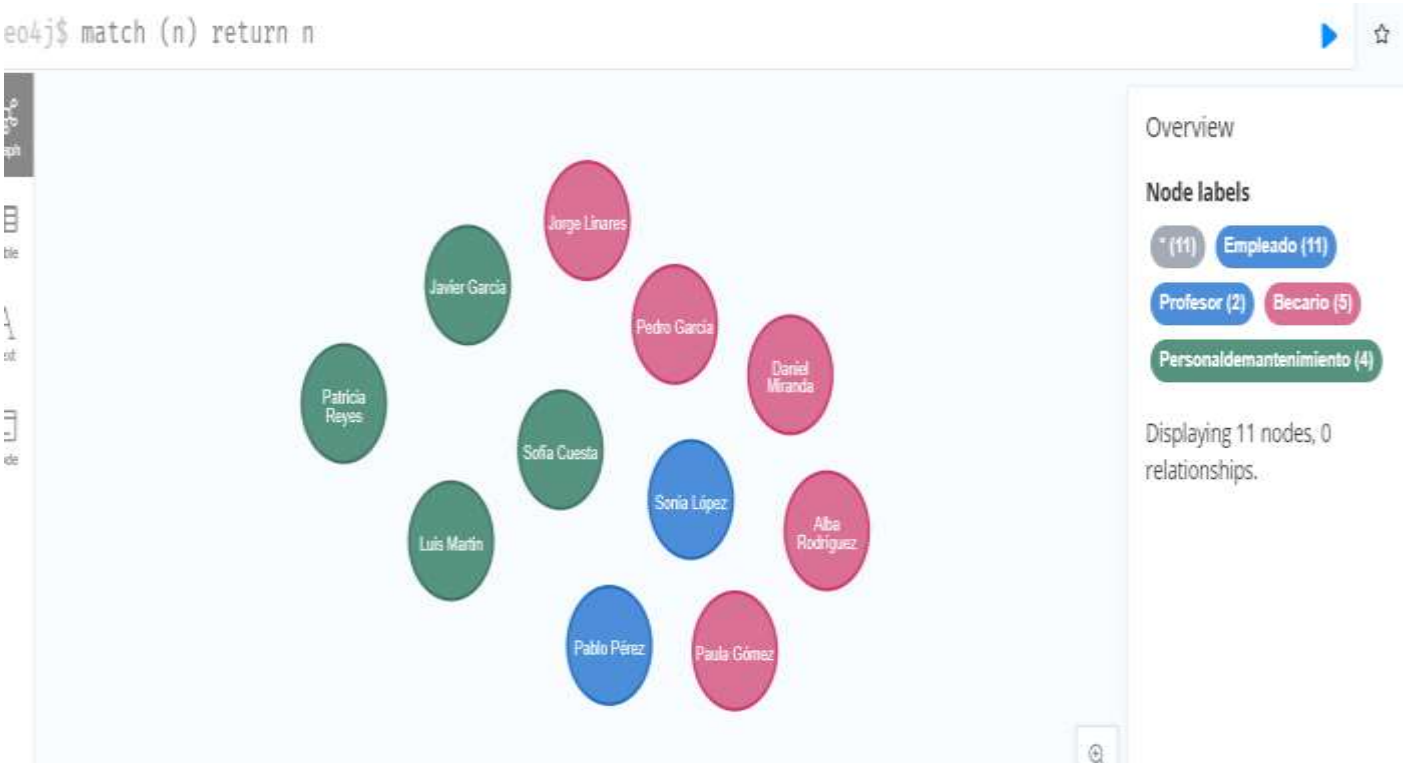


Figura 34 Representación grafo jerarquía Neo4J

Con esta transformación también se conserva la semántica, ya que dentro del nodos subtipos (Profesor, Becario, Personal de mantenimiento) estamos guardando la información del empleado con la plaza, además seguimos asegurando que un empleado solo pueda ocupar una plaza.

En cuanto al tipo de jerarquía (parcial) el sistema nos va a permitir insertar empleados de otros tipos, para recoger que se trata de una jerarquía parcial. Por otro lado, como un empleado mantiene las restricciones de unicidad no permitiendo crear dos veces el mismo empleado, no podemos crear el mismo empleado perteneciendo a dos subtipos por tanto esto hace que la jerarquía sea exclusiva.

En la Figura 35 vamos a intentar insertar un empleado que es de tipo profesor como becario para demostrar que Neo4J muestra un error, con lo que se recoge la semántica de la jerarquía exclusiva y no se permite crear un empleado ya existente.



Figura 35 Error grafo jerarquía

En resumen, jugando con la restricción de los atributos con la transformación propuesta, podemos tener una jerarquía tanto solapada como exclusiva. El problema semántico de esta guía es que solo podemos tener una jerarquía parcial, ya que no tenemos una restricción que impida crearnos otra etiqueta que sea otro tipo de empleado.

3.3.3 Transformación de la Interrelación 1:N - Imparte

En este apartado vamos a representar cómo se transforma la interrelación 'Imparte' entre profesores y asignaturas, aplicando las guías propuestas.

Aplicando la guía para esta interrelación el esquema conceptual resultado en Neo4J es el visto en la Figura 36. Vemos como el nodo 'Profesor' y el nodo 'Empleado' están relacionados con el nodo 'Asignatura' mediante una arista llamada 'imparte'. Destacamos que la arista que hemos creado parte desde el nodo 'Profesor' con dirección al nodo 'Asignatura', aunque no tengamos bidireccionalidad esto no es un problema a la hora de hacer consultas.



Figura 36 Representación de interrelación 1:N en Neo4J

Hemos realizado la carga de un par de empleados impartiendo sus asignaturas para el caso de estudio. En la Figura 37 vemos como queda en Neo4J.

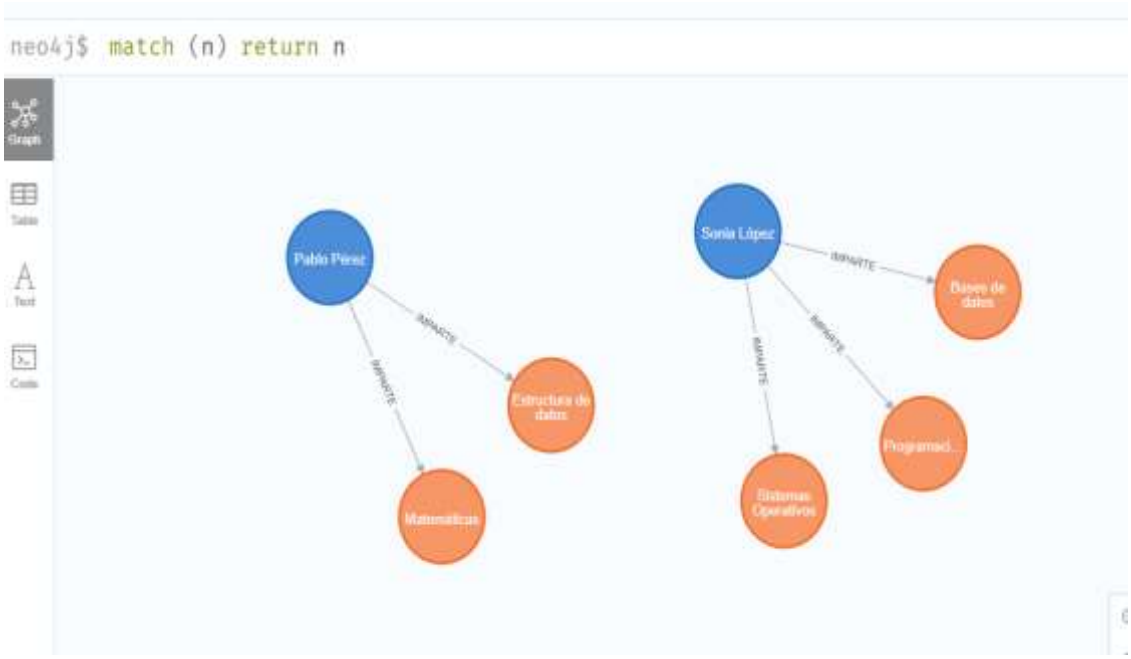


Figura 37 Representación de interrelaciones existentes entre empleados y asignaturas en Neo4J

En cuanto a la semántica el profesor debe ser único manteniendo las restricciones del empleado, en cuanto a la asignatura nos definimos en las restricciones de la Figura 38 que el nombre de la asignatura debe ser único para que no se repitan las asignaturas, además de definir el curso como obligatorio.

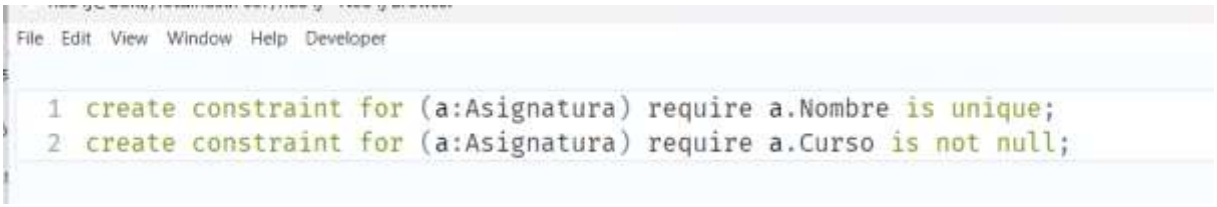


Figura 38 Representación restricciones asignaturas

Sin embargo, en este caso sí perdemos semántica, ya que podemos crearnos instancias de profesores sin que tengan la arista apuntando a la asignatura. Esto hace que no podamos cumplir la cardinalidad de que un profesor imparta como mínimo una asignatura.

De la misma manera puede ocurrir que tengamos asignaturas creadas que no son impartidas por profesores, por tanto, con esta guía la semántica que perdemos principalmente parte de la cardinalidad.

Además, vemos como Neo4J admitiría que una asignatura pueda ser impartida por dos profesores por tanto esa parte tampoco conseguimos dejarla semánticamente correcta.

3.3.4 Transformación de la Interrelación N:M - Trabaja

En este apartado aplicamos la regla de transformación propuesta para este tipo de interrelación N:M.

Tras aplicar la transformación propuesta vemos en la Figura 39 el esquema lógico resultante en Neo4J: Se creará un nodo 'Personal de mantenimiento' que apunta al nodo 'Campus'. Aunque no sea posible crear una relación bidireccional en Neo4J, sí que es posible recorrer las relaciones en los dos sentidos, tal y como veremos en las consultas propuestas.

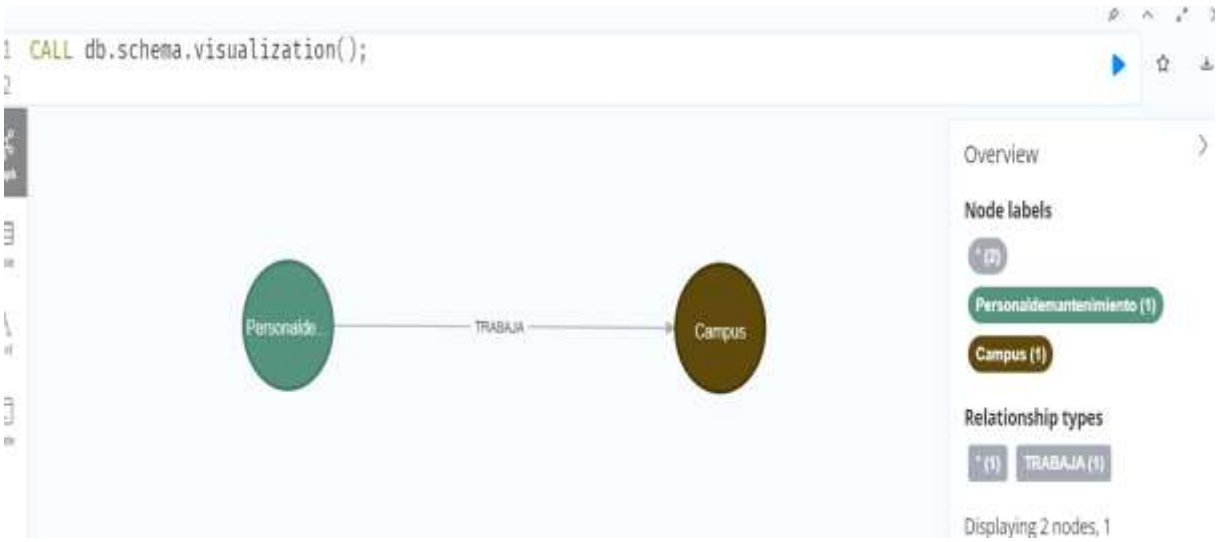


Figura 39 Representación de interrelación N:M en Neo4J

Seguidamente hacemos la carga de datos para estos nodos, cargando los campus (marrón) y las relaciones con sus respectivos empleados (verde), ver Figura 40.

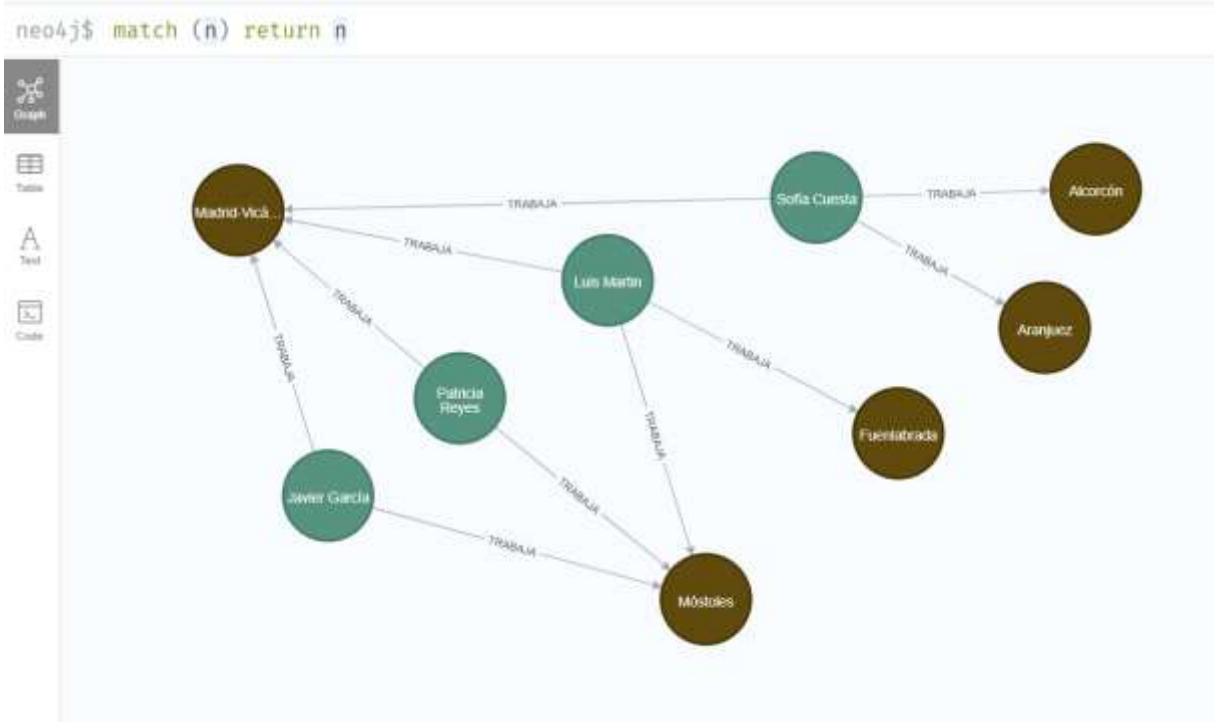


Figura 40 Representación de interrelaciones existentes entre Personal de mantenimiento y Campus en Neo4J

En cuanto a las restricciones por parte del personal de mantenimiento mantiene las restricciones del empleado. En cuanto al campus definimos las restricciones de la Figura 41 para asegurar que no existan campus repetidos.

```
File Edit View Window Help Developer
1 CREATE CONSTRAINT for (c:Campus) require c.cod_campus is unique;
2 CREATE CONSTRAINT for (c:Campus) require c.Ciudad is not null;
3
```

Figura 41 Restricción campus Neo4J

En cuanto a la semántica vemos cómo mantenemos con esta transformación de forma correcta los empleados y los campus, con sus respectivas restricciones. Además, al ser una interrelación N:M, se mantiene la semántica de que un empleado se puede relacionar con varios campus y en un campus puedan trabajar varios empleados.

También mantenemos el atributo fecha de la interrelación el cual es un atributo de la arista ‘trabaja’.

El único caso que no podemos recoger en la semántica, es el caso de que un empleado deba trabajar obligatoriamente en un campus. Si creamos una ocurrencia de un personal de mantenimiento, el sistema lo va a permitir sin la necesidad de crear una arista hacia campus. Comentar que si la cardinalidad fuera de (0,n) entonces sería correcto, pero si lo fijamos en (1,n) no podemos asegurar que un profesor trabaje como mínimo en un campus.

3.3.5 Transformación de la Interrelación Reflexiva - Es_Compañero_De

Aplicamos la guía de transformación para las interrelaciones reflexivas. En este caso el esquema conceptual en Neo4J visto en la Figura 42 contiene el nodo 'Becario' con la arista apuntándose a sí mismo.



Figura 42 Representación de interrelación reflexiva en Neo4J

Hemos cargado la relación entre varios becarios de los ya creados en nuestra base de datos. En la Figura 43 se muestra como quedan los datos en Neo4J.

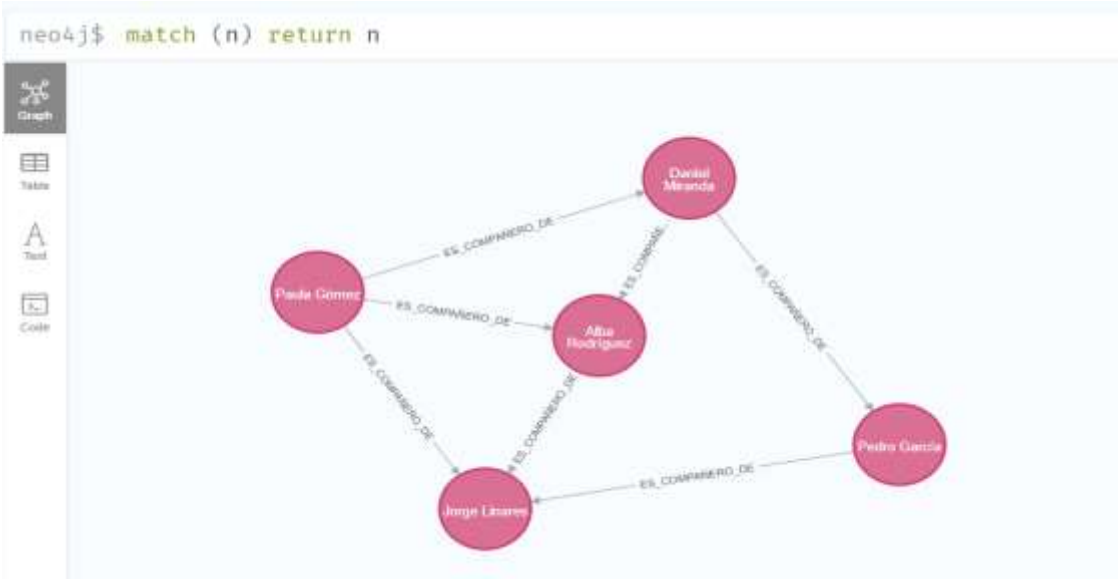


Figura 43 Representación de interrelaciones existentes entre Becarios en Neo4J

Como hemos visto este tipo de interrelación se puede realizar en Neo4J creando aristas entre los nodos del mismo tipo.

En cuanto a la semántica los empleados siguen sometidos a la restricción de unicidad, de nuevo lo único que perdemos es la condición de que un becario deba tener obligatoriamente un compañero.

3.3.6 Esquema final y consultas

Al realizar la transformación completa del esquema conceptual aplicando las guías propuestas, vemos en la Figura 44 la representación completa del esquema en Neo4J, donde ya tenemos representados todos los nodos con sus respectivas aristas para definir las relaciones. En la parte derecha vienen los tipos de nodos que tenemos en el grafo, y las aristas o relaciones existentes.

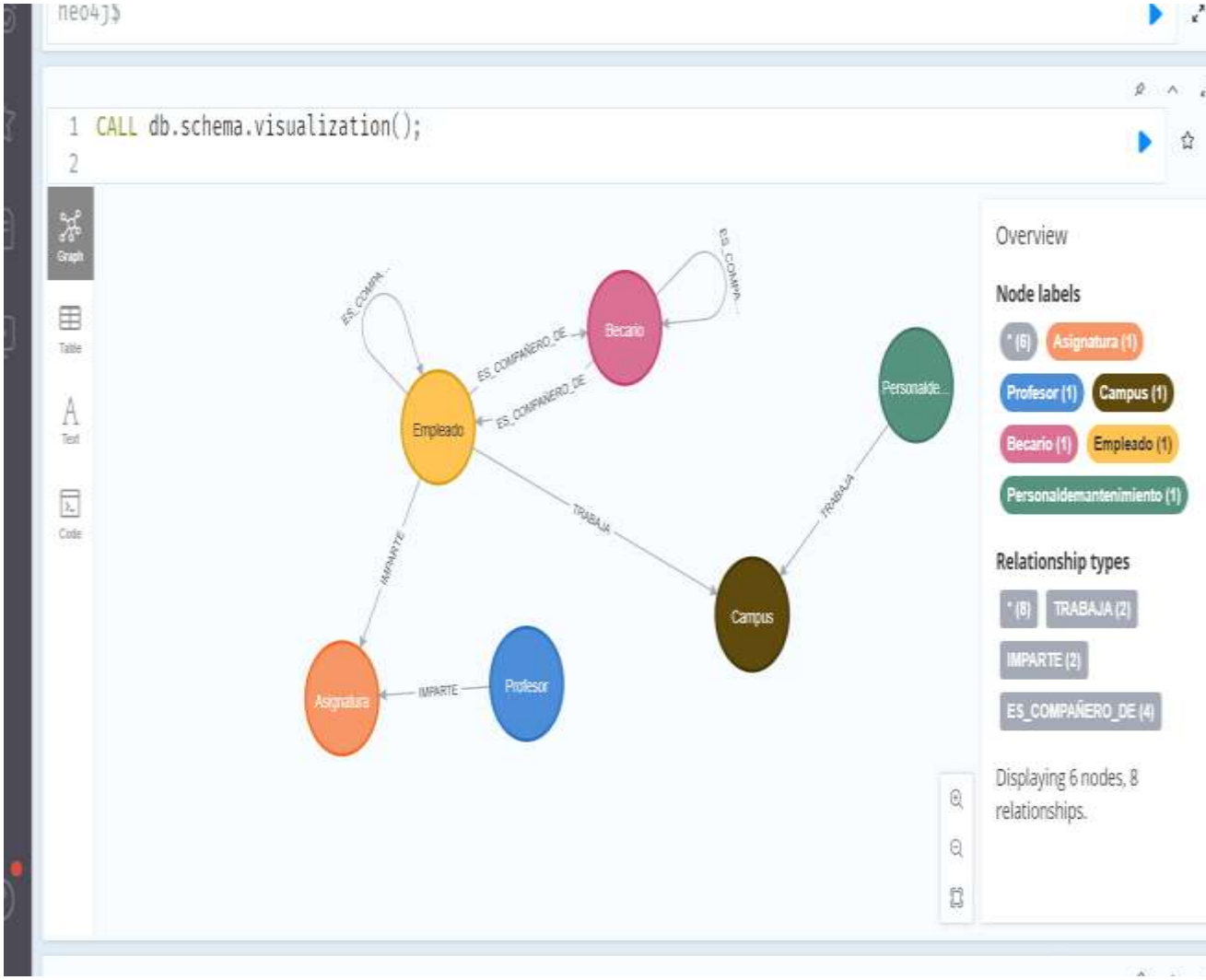


Figura 44 Representación esquema completo

Como hemos hecho la carga de todos los datos, en la Figura 45 se muestra el grafo que recoge todos los datos de ejemplo insertados en nuestra base de datos, donde cada nodo viene representado del color correspondiente a su etiqueta.

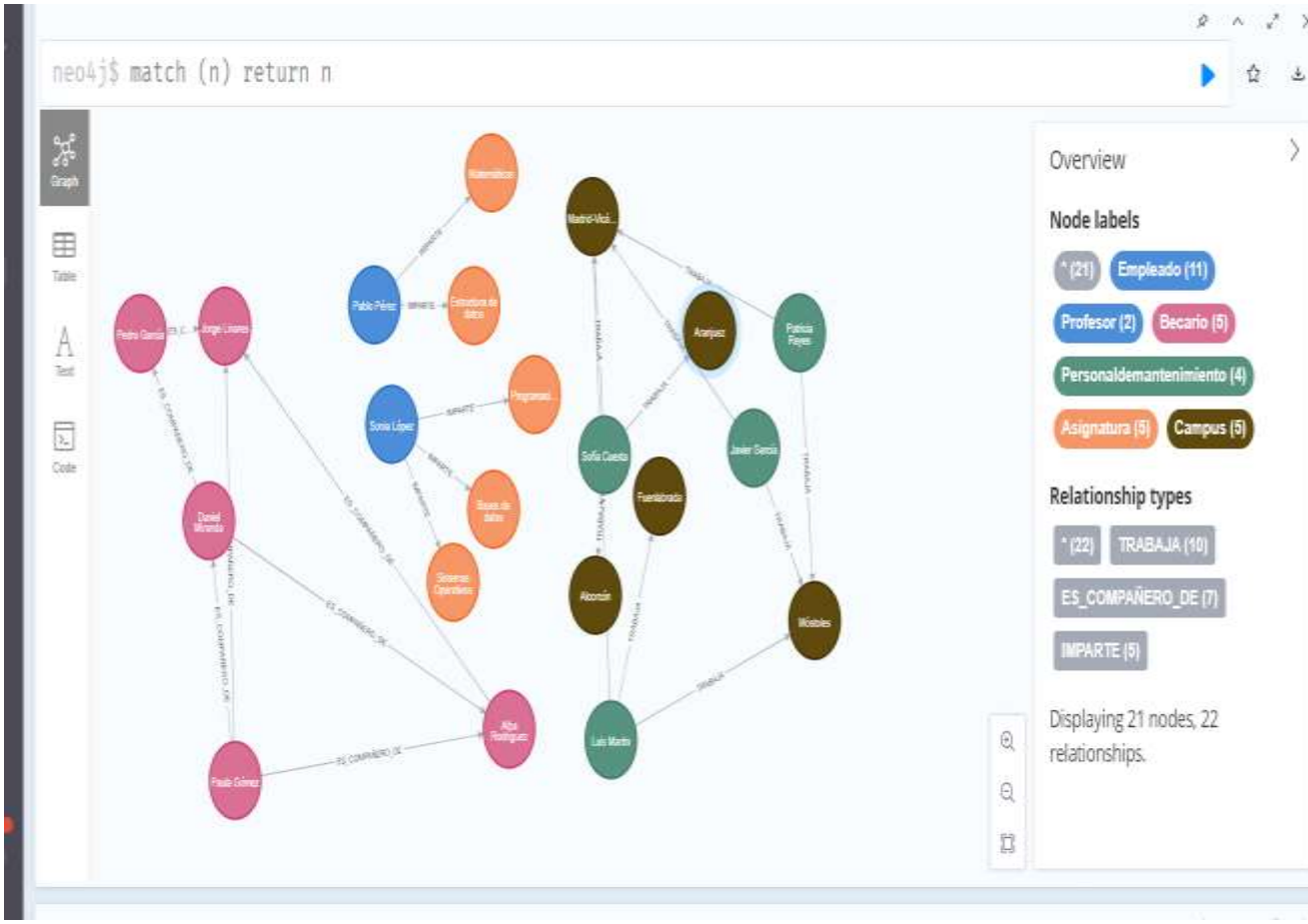


Figura 45 Representación grafo completo

Como resumen en cuanto a semántica no recogida en la transformación, vamos a poder tener más empleados de otros tipos sin la posibilidad de tener una jerarquía total debido a que hemos perdido el atributo tipo de la jerarquía, sin este atributo no podemos elegir el tipo de jerarquía. Además, vamos a poder instancias de empleado que no tengan ninguna arista, con lo que podríamos tener nodos sueltos sin ningún tipo de relación en nuestro grafo. En general no estamos consiguiendo recoger la cardinalidad mínima de las interrelaciones, en todas ellas podemos tener instancias sin relacionar sin poder recoger las cardinalidades (1,n), ya que estamos recogiendo cardinalidades (0,n)

Por último, vamos a realizar una serie de consultas para poder verificar el funcionamiento y la semántica de nuestra base de datos:

- **Primera consulta:** Vamos a sacar todas las asignaturas de tercer curso impartidas por profesores que tengan menos de 40 años.

Queremos demostrar que partiendo de la asignatura podemos llegar a comprobar los profesores, aunque no tengamos una arista que parta de la asignatura con destino profesor.

Como se observa en la Figura 46, tenemos en Cypher la consulta con la que rescatamos las asignaturas impartidas por profesores menores de 40 años. Se aprecia cómo a la hora de hacer la consulta, seguimos relacionando al profesor con la asignatura con la misma dirección y filtramos por la edad. Se verifica que en Neo4J no importa la dirección de la arista, rescatando de forma correcta las asignaturas.

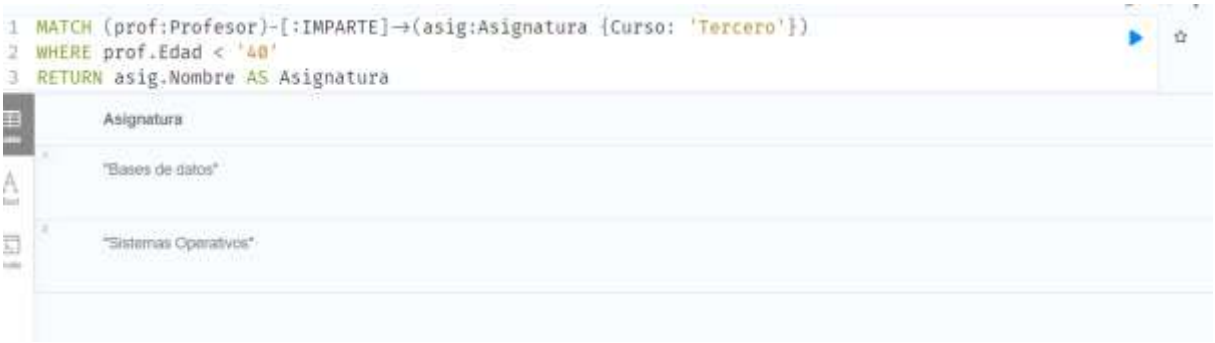


Figura 46 Ejecución de la consulta 1

- **Segunda consulta:** la consulta que haremos es obtener todos los campus en los cuales trabajen empleados que estén asociados a una plaza sin descripción y que la fecha de ocupación de la plaza sea anterior al año 2021 para demostrar que podemos relacionar cualquier dato de la base datos.

En la Figura 47 vemos como queda la consulta en Neo4J, viendo cómo desde un campus podemos llegar a relacionarlo con la plaza del empleado. Destacar el uso de la cláusula DISTINCT para evitar campus duplicados ya que puede ocurrir que dos empleados que cumplan la condición apunten al mismo campus.

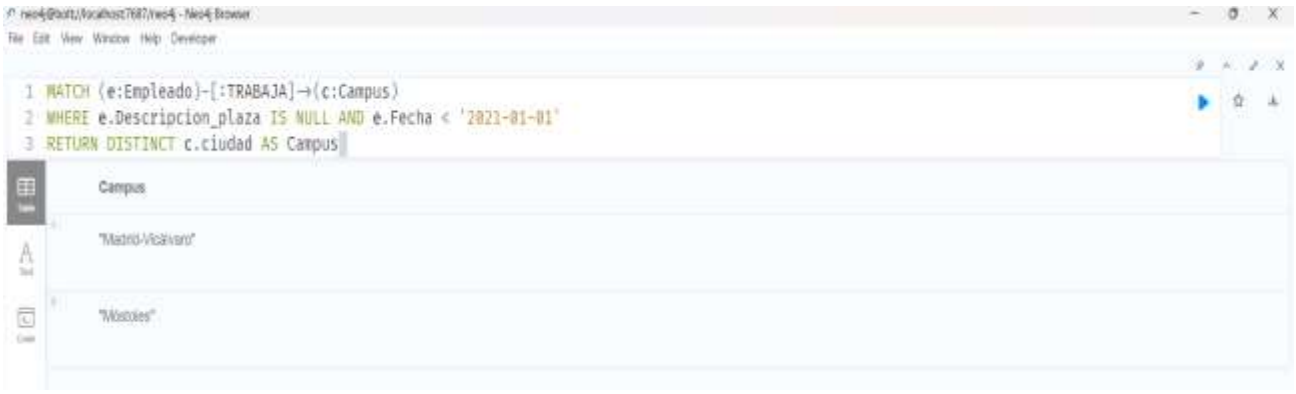


Figura 47 Ejecución de la consulta 2

- **Tercera consulta:** Queremos verificar que la interrelación reflexiva funciona correctamente, haremos una consulta para saber el número de becarios que tenemos que su fecha de ocupación de la plaza sea posterior a 2020 y además tengan al menos tres compañeros becarios, así comprobamos que el becario se puede relacionar con la plaza y observamos si se verifica la interrelación reflexiva.

En la Figura 48 vemos como saca de forma correcta el número de becarios que cumplen estas condiciones.



Figura 48 Ejecución de la consulta 3

- **Cuarta consulta:** Vamos a intentar realizar una consulta que intente pasar por todos los nodos y relaciones creadas. Queremos saber el nombre y la descripción de las plazas ocupadas por profesores que impartan más de dos asignaturas, o bien que trabaje en un campus posterior al 2020, o que sea del grado alimentario.

Como vemos en la Figura 49 en la consulta podemos sacar la información recorriendo todas los nodos y las aristas de nuestro grafo.



Figura 49 Ejecución de la consulta 4

3.4 Lecciones aprendidas

En este punto hablaremos de las lecciones aprendidas durante el desarrollo del trabajo destacando aquí los principales problemas encontrados.

En primer lugar, aunque tenemos mucha información del funcionamiento de las bases de datos orientadas a grafos, en cuanto a guías de diseño no hemos encontrado trabajos en los que se desarrolle una metodología de diseño de este tipo de base de datos, por lo que la principal dificultad del trabajo ha sido idear unas guías propias. Esto hace que haya habido propuestas iniciales que a la hora de verificarlas en Neo4J, no se corresponde con lo deseado y se hayan tenido que desechar.

Además, otro punto importante es el estudio de la semántica, con las guías propuestas he intentado mantener la mayor parte de la información del esquema conceptual, lo cual es complicado. Es cierto que, mediante guías y restricciones, se puede mantener la gran parte de la semántica, pero no siempre es conservar la totalidad de la semántica.

Otro problema encontrado ha sido que, al ir troceando distintas interrelaciones, a la hora de hacer la validación quedan entidades y relaciones sin cuadrar como realmente teníamos pensado. Por lo tanto, la mejor manera de hacer el diseño completo de la base de datos orientada a grafos es pensar en todo el esquema conceptual sin trocear. Sin embargo, a la hora de explicar y establecer guías para ejemplos más desarrollados, es más intuitivo explicarlo por partes.

IV. CONCLUSIONES Y FUTUROS TRABAJOS

En este Trabajo Fin de Grado hemos definido unas guías de transformación para realizar el diseño de una base de datos orientada a grafos a partir de un esquema conceptual con el objetivo de conservar la mayor semántica posible. Para ello en primer lugar nos hemos familiarizado con este tipo de BD NoSQL para luego, a partir de un caso de estudio sencillo poder definir las guías de transformación. Se han aplicado las reglas al caso de estudio, se han cargado los datos en Neo4J usando el lenguaje Cypher y se han desarrollado una serie de consultas para validar la propuesta. Con esto podemos concluir que se ha alcanzado satisfactoriamente el objetivo propuesto inicialmente.

Como conclusión global, hemos podido comprobar que las bases de datos orientadas a grafos son muy atractivas para aplicaciones web con datos muy relacionados, máxime si el volumen de datos es alto.

Personalmente, me ha resultado instructivo estudiar las bases de datos orientadas a grafos. Vistas las distintas posibilidades de transformación, he podido observar que no existe una única alternativa ideal, ya que depende del tipo información que estemos manejando y también de las consultas más frecuentes. Por ello, podemos decir que las bases de datos orientadas a grafo son muy recomendables para casos con un gran volumen de datos muy relacionado, al permitir una gran flexibilidad y un alto rendimiento en las consultas.

Como futuros trabajos se podría profundizar más en las guías propuestas, exponiendo ejemplos más complejos y cercanos al mundo real. Un punto muy interesante sería probar nuestras guías para medir la eficiencia y comparar con otras posibles guías para poder elegir por su eficiencia además de por su semántica.

V. REFERENCIAS BIBLIOGRÁFICAS

1. Andreas Meier, Michael Kaufmann. 'SQL & NoSQL Databases: Models, Languages, Consistency Options and Architectures for Big Data Management'. Springer Vieweg. 29 Agosto 2019.
2. Thomas Frisendal. 'Visual Desing of GraphQL Data'. Apress. 9 Septiembre 2018
3. Pramod Saladage, Martin Fowler, 'NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistance'. Addison-Wesley Professional. 8 Agosto 2012.
4. BD NoSQL ventajas y productos. 'https://www.arsys.es/blog/bases-de-datos-nosql-que-son-tipos-y-ventajas'.
5. Dan Sullivan, 'NoSQL for Mere mortals'. Addison-Wesley Professional 16 Abril 2015.
6. Jim Webber, Ian Robinson, Emil Eifren. 'Graph Databases'. O'Really Media. 17 Junio 2013.
7. Cristina Pérez Sola, Jordi Conesa i Caralt, M. Elena Rodríguez González. '¿Cómo usar una base de datos en grafo? UOC, S.L. 6 Marzo 2016.
8. Documentación Neo4J 'https://neo4j.com/docs/'.